

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Best Bike Paths (BBP) Requirements Analysis and Specification Document (RASD)

Software Engineering II

Advisor: Prof.ssa Elisabetta Di Nitto
Academic Year: 2025-2026
Students:

- Giuseppe
- Lorenzo
- Giacomo

Deliverable:	RASD
Title:	Requirements Analysis and Specification Document
Authors:	Giuseppe, Lorenzo, Giacomo
Version:	1.0
Date:	2025-12-23
Download page:	`https://github.com/giusedmb/best-bike-paths-se2`
Copyright:	Copyright © 2025 – All Rights Reserved

Contents

Contents **i**

List of Figures **iii**

List of Tables **v**

1 Introduction **1**
1.1 Purpose . . . 1
1.1.1 Goals . . . 2
1.2 Scope . . . 3
1.2.1 World Phenomena . . . 3
1.2.2 Shared Phenomena . . . 4
1.3 Definitions, Acronyms, and Abbreviations . . . 6
1.4 Revision History . . . 7
1.5 Reference Documents . . . 7
1.6 Document Structure . . . 7

2 Overall Description **8**
2.1 Product Perspective . . . 8
2.1.1 Scenarios . . . 8
2.1.2 Class Diagram . . . 10
2.2 Product Functions . . . 13
2.3 User Characteristics . . . 22
2.4 Assumptions, Dependencies and Constraints . . . 22
2.4.1 Assumptions . . . 22
2.4.2 Dependencies . . . 23
2.4.3 Constraints . . . 23

3 Specific Requirements **25**

3.1 External Interface Requirements . . . 25
3.1.1 User Interfaces . . . 25
3.1.2 Hardware Interfaces . . . 26
3.1.3 Software Interfaces . . . 26
3.1.4 Communication Interfaces . . . 26
3.2 Functional Requirements . . . 27
3.2.1 Requirements List . . . 27
3.2.2 Use Case Diagrams . . . 29
3.2.3 Use Cases . . . 31
3.2.4 Goals - Requirements Mapping . . . 53
3.3 Performance Requirements . . . 55
3.4 Design Constraints . . . 55
3.4.1 Standards Compliance . . . 55
3.4.2 Hardware Limitations . . . 56
3.5 Software System Attributes . . . 57
3.5.1 Reliability . . . 57
3.5.2 Availability . . . 57
3.5.3 Security . . . 57
3.5.4 Maintainability . . . 57
3.5.5 Portability . . . 57

4 Formal Analysis using Alloy 58
4.1 Alloy Model . . . 58
4.1.1 Signatures and Enums . . . 58
4.1.2 Domain Entities: Trips, Weather, and Reports . . . 59
4.1.3 System Facts and Constraints . . . 61
4.1.4 Assertions and Predicates . . . 64
4.2 Alloy Results . . . 66

5 Effort Spent 72

6 References 73
6.1 References . . . 73
6.2 Tools and Technologies . . . 73

List of Figures

2.1 High Level View - Class Diagram 10
2.2 Account Creation - Product Function 14
2.3 Edit profile - Product Function 15
2.4 Publish Trip - Product Function 17
2.5 Automatic Trip - Product Function 18
2.6 Manual Trip and Edit Trip - Product Function 19
2.7 Search Trip - Product Function 20
2.8 Weather Conditions Retrieval - Client-side Subprocess 21

3.1 Unregistered User abilities - Use Case Diagram 29
3.2 Registered Cyclist abilities - Use Case Diagram 30
3.3 System and external services - Use Case Diagram 31
3.4 Cyclist registration - Sequence Diagram 33
3.5 Cyclist Login - Sequence Diagram 35
3.6 Start automatic trip recording - Sequence Diagram 37
3.7 Start manual trip recording - Sequence Diagram 39
3.8 View trip history and trip details - Sequence Diagram 41
3.9 Submit obstacle report - Sequence Diagram 43
3.10 Validate detected obstacle - Use Case Diagram 45
3.11 Submit obstacle report - Sequence Diagram 47
3.12 Submit obstacle report - Sequence Diagram 50
3.13 Manage data publishability - Use Case Diagram 52
3.14 Manage data publishability - Sequence Diagram 53

4.1 Generated image for the `showAutomaticTrip` predicate 67
4.2 Obstacle workflow instance showing the coexistence of Pending, Confirmed, and Rejected reports, where only the confirmed report is linked to a physical Obstacle. 68
4.3 Path Merging instance showing `BikePath2` resolving a status of MEDIUM based on the majority of incoming user reports. 69

4.4 Complete scenario instance showing the coexistence of automatic and manual trips, obstacle reporting with different validation states, path status aggregation, and environmental conditions. 70
4.5 Majority voting scenario in which two `OPTIMAL` reports override a single `MEDIUM` report, resulting in an aggregated `OPTIMAL` status for the bike path. 71

List of Tables

1.1 Goals Table for Best Bike Paths (BBP) 2
1.2 World Phenomena Table for Best Bike Paths (BBP) 3
1.3 Shared Phenomena Table for Best Bike Paths (BBP) 5
1.4 Definitions, Acronyms, and Abbreviations Table 6

2.1 Domain Assumptions Table 23
2.2 Domain Dependencies Table 23
2.3 Domain Constraints Table 24

3.1 System Requirements Table 28
3.12 Goals to Requirements Mapping Table 55

5.1 Effort Spent by Each Member of the group 72

1 | Introduction

Best Bike Paths (BBP) is a software system designed to support cyclists in creating, exploring, and maintaining an organized inventory of bike paths. The platform enables registered users to record their trips, monitor their cycling activities, and access meaningful statistics such as total distance, average speed, and performance metrics. When available, trip information can be enriched with meteorological data retrieved from external services, providing users with a more complete overview of their routes.

In addition to trip management, BBP allows users to manually insert information about bike paths, including their condition and the presence of obstacles such as potholes, debris, or other hazards. The system also offers an automated mode for acquiring data through mobile sensors (e.g., GPS, accelerometer, gyroscope), with a confirmation step to ensure reliability.

This document provides a comprehensive description of BBP's capabilities, constraints, and design decisions. Its purpose is to serve as a detailed reference for the development team and as a clear source of information for all stakeholders involved, including cyclists, associations, and maintenance authorities.

1.1. Purpose

This document primarily serves as a comprehensive reference for the development team responsible for implementing Best Bike Paths (BBP).

Moreover, this material aims to provide an authoritative overview of the system's functionalities and constraints, ensuring a clear and shared understanding among all stakeholders, particularly the cyclists and administrative bodies.

1.1.1. Goals

Goal	Description
From a registered user perspective...	
G01	Allow the RU to create and manage a personal account.
G02	Allow the RU to record private personal trips (manual or automatic).
G03	Allow the RU to view detailed statistics for each trip (distance, speed, performance metrics).
G04	Allow the RU to manually insert path information by selecting streets and specifying their status.
G05	Allow the RU to collect automatic trip data via mobile sensors (GPS, accelerometer, gyroscope).
G06	Allow the RU to confirm or correct automatically inferred obstacles or street conditions.
G07	Allow the RU to publish, edit, or delete paths they contributed.
G08	Allow the RU to see a dashboard summarizing their cycling activity.
From any user (registered or casual)...	
G09	Allow users to browse all published paths and their associated conditions.
G10	Allow users to search for paths based on origin and destination.
G11	Allow users to view map representations of possible routes.
G12	Allow users to sort or filter paths based on score or conditions.
From a system perspective...	
G13	Enable the system to compute the score of a trip/path based on its characteristics.
G14	Enable the system to enrich trips with meteorological information from an external weather service.
G15	Enable the system to detect anomalies such as potholes using sensor data.
G16	Enable the system to merge similar paths contributed by multiple users, considering freshness and disagreement handling.
G17	Enable the system to visualize map data using a mapping API (e.g., OpenStreetMap).

Table 1.1: Goals Table for Best Bike Paths (BBP)

1.2. Scope

This section defines the boundaries of the Best Bike Paths (BBP) system by analyzing the real-world context in which it operates and the interactions between the system and external entities. We introduce the relevant world phenomena and the shared phenomena that represent the exchange of information between the system and its environment. Together, these elements clarify what belongs to the system and how the two domains interact.

1.2.1. World Phenomena

Phenomena	Description
WP01	Users bike through real-world streets and bike paths.
WP02	Users perceive and evaluate street status (optimal, medium, sufficient, requires maintenance, not available).
WP03	Obstacles occur in the real world (potholes, leaves, snow, rain, animals).
WP04	Mobile-device sensors detect real-world movement (GPS position, acceleration, rotation).
WP05	Meteorological conditions exist in the real world (temperature, wind speed, weather events).
WP06	Different users perceive the same street with different conditions, causing conflicting data.
WP07	Users select real-world points as origin and destination for routing.

Table 1.2: World Phenomena Table for Best Bike Paths (BBP)

1.2.2. Shared Phenomena

Explanation of Controller and Observer

In the context of problem frames, each shared phenomenon involves two entities: the one that *controls* the phenomenon and the one that *observes* it. This distinction clarifies how the interaction between the world and the machine is structured.

- **Controller**: the domain or component that *causes* the phenomenon to occur. It is the entity that initiates, modifies, or generates the event. For example, when a registered user records a trip or inserts manual information about a bike path, the user is the *controller* of the phenomenon. Conversely, when the system computes statistics or merges data from multiple users, the system (BBP) is the controller.
- **Observer**: the domain or component that *perceives* the phenomenon once it has occurred. It receives or reacts to the outcome of the event but does not generate it. For instance, when the user submits new information, the system observes it. When the system displays possible bike paths or computed scores, the user is the observer.

This distinction helps classify each phenomenon as world-controlled, machine-controlled, or shared, and makes explicit the direction of information flow between the user and the system.

Phenomena	Description	Controller	Observer
SP01	Registered users create or modify their personal trips.	RU	BBP
SP02	BBP stores trip information and computes trip statistics.	BBP	RU
SP03	Registered users manually insert street segments and their status.	RU	BBP
SP04	BBP saves manual path contributions and makes them publishable.	BBP	RU
SP05	Sensor data (GPS, accelerometer, gyroscope) is sent from the device to BBP.	RU	BBP
SP06	BBP infers anomalies (potholes, obstacles) and shows them to the user for confirmation.	BBP	RU
SP07	Users confirm or reject automatically inferred anomalies.	RU	BBP
SP08	Users publish, edit, or delete their previously contributed paths.	RU	BBP
SP09	Any user requests paths between origin and destination.	User	BBP
SP10	BBP displays path options and their score on a map.	BBP	User
SP11	The Weather Service provides meteorological information to BBP via API.	WeatherService	BBP
SP12	BBP merges path information from different users into a unified representation.	BBP	Users

Table 1.3: Shared Phenomena Table for Best Bike Paths (BBP)

1.3. Definitions, Acronyms, and Abbreviations

Acronym	Definition
BBP	Best Bike Paths, the software system described in this document, aimed at supporting cyclists in recording trips, managing bike paths, and sharing information about street conditions.
RASD	Requirements Analysis and Specification Document, the document that describes requirements, constraints, and domain assumptions of the system.
RU	Registered User, an authenticated user of BBP who can record trips, contribute path information, and publish or update bike paths.
GPS	Global Positioning System, used to determine the geographic position of a device during trip recording.
API	Application Programming Interface, an external service interface used by BBP to retrieve map data or meteorological information.
WPx	World Phenomenon number x, representing a real-world event or condition outside the system boundary but relevant to the problem domain.
SPx	Shared Phenomenon number x, representing an interaction between the system and external actors, involving information exchange.
Path	A bike route composed of one or more street segments, defined either manually by users or derived from recorded trips.
Street	A single road segment that is part of a bike path and can be associated with condition reports and obstacles.
Trip	A cycling activity recorded by a registered user, containing temporal, spatial, and performance data, acquired either manually or automatically.
TrackPoint	A single sampled point of a recorded trip, containing geographic coordinates, timestamp, and optional elevation data.
PathStatus	An enumeration representing the condition of a street or path, such as optimal, medium, sufficient, or requiring maintenance.
QoE	Quality of Experience, a measure of how suitable or comfortable a bike path is perceived to be by users, considering safety, conditions, and usability.
GDPR	General Data Protection Regulation, the European regulation governing the processing and protection of personal data.
EU	European Union, the legal and regulatory context in which BBP is assumed to operate.

Table 1.4: Definitions, Acronyms, and Abbreviations Table

1.4. Revision History

- **Version 1.0** - Initial release of the Requirements Analysis and Specification Document.

1.5. Reference Documents

- Specification document: R&DD Assignment AY 2025-2026.
- Provided course materials and notes (Politecnico di Milano).
- Alloy Documentation.

1.6. Document Structure

This document is structured to provide a clear and progressive understanding of the Best Bike Paths (BBP) system, starting from high-level concepts and gradually introducing more detailed analyses and specifications.

Chapter 1 introduces the purpose of the system, its main goals, and the overall scope of the problem. It defines the relevant world and shared phenomena and clarifies the boundaries between the system and its environment.

Chapter 2 provides an overall description of BBP, including representative usage scenarios and the high-level domain model. This chapter helps the reader understand how different actors interact with the system and how the main entities relate to each other.

The subsequent chapters focus on the detailed requirements of the system. Functional requirements are presented through use cases and scenarios, while non-functional requirements specify quality attributes, constraints, and design assumptions.

Formal aspects of the system are addressed in the chapter dedicated to formal analysis, where key components and processes are modeled using Alloy in order to verify consistency, correctness, and critical properties of the system.

Finally, the document concludes with reference materials and a revision history, ensuring traceability and supporting future updates of the specification.

2 Overall Description

2.1. Product Perspective

2.1.1. Scenarios

Scenario 1: Registered User Creates an Account and Records a Trip Emma R. is a regular cyclist who wants to keep track of her personal bike activity. She installs the Best Bike Paths (BBP) mobile application and creates an account by providing basic identification and contact information. Once her account is verified, she can access the app's personalised features.

During her daily commute, Emma decides to record her ride. She starts a new trip and BBP automatically collects GPS data, speed, and performance-related statistics. At the end of the route, the system stops the recording and stores the trip in her private dashboard. BBP also retrieves contextual meteorological information (temperature, weather conditions) for that time interval from an external service. Emma can later review her rides and aggregated statistics through her personal area.

Scenario 2: Manual Definition and Publication of a Bike Path Luca P. is an experienced cyclist who wants to publish a safe route that is not well documented in existing map services. He accesses BBP with his registered account and uses the manual path creation feature to define a custom bike route.

BBP provides map-based tools and a street search mechanism that allow Luca to select the relevant street segments and assign each segment a status (such as optimal, medium, or requiring maintenance). He may also optionally indicate obstacles identified along the route. Once satisfied with the defined path, he assigns it a name and publishes it. The system computes a score for the new route based on the provided data, making it available to all users performing searches in that area.

Scenario 3: Automatic Trip Recording with Sensor-Based Obstacle Detection Sara C. wants to contribute information about the conditions of a road she frequently uses. She activates BBP's automatic trip recording, which uses GPS data together with avail-

able mobile sensors (such as accelerometer and gyroscope) to detect potential anomalies along the route.

During her ride, the system identifies unusual vibrations or irregularities that might correspond to obstacles. After the trip ends, BBP presents Sara with a summary of the detected points and asks her to confirm or correct the interpretation of the obstacles, ensuring false positives are handled. Once validated, the information is stored and may enrich the quality of future published paths and their scores.

Scenario 4: Casual User Searches for a Path Between Two Locations Marco L. is visiting a city for a weekend and wants to reach a park starting from his hotel using a safe bike route. Without creating an account, he accesses BBP as a casual user and specifies his starting point and destination.

BBP identifies the available routes matching the query and ranks them according to their score, which reflects factors such as route condition and overall status. Marco inspects the proposed alternatives and visualises the most suitable route on the map. Even without registering, he can use BBP to safely reach his destination.

Scenario 5: Registered User Publishes and Later Updates a Frequently Used Path After several weeks of using BBP, Emma R. notices that her recurring commuting route is not available as a public path. She selects one of her recorded private trips and uses BBP's functionality to derive a candidate publishable path. She verifies the correctness of the route and adjusts the status of certain segments based on her experience. After assigning a name to the path, she publishes it, making it available to other users.

Scenario 6: User Updates an Existing Path with New Data Tom H. has recently moved to the city and uses BBP to find a reliable bike route between his home and the train station. While following the suggested route, he starts recording his trip through the application. Once the ride is completed, he reviews the automatically detected information and assigns the road status as optimal. In the BBP database, that same path was previously rated as medium by other users. Since Tom provides publishable information, BBP updates the stored data by incorporating his contribution. The system re-evaluates the score of the path by considering the freshness of Tom's input and the number of previous submissions. If Tom's assessment is consistent with other recent reports, the aggregated status is adjusted accordingly; otherwise, the system keeps the data until further contributions resolve the conflict.

2.1.2. Class Diagram

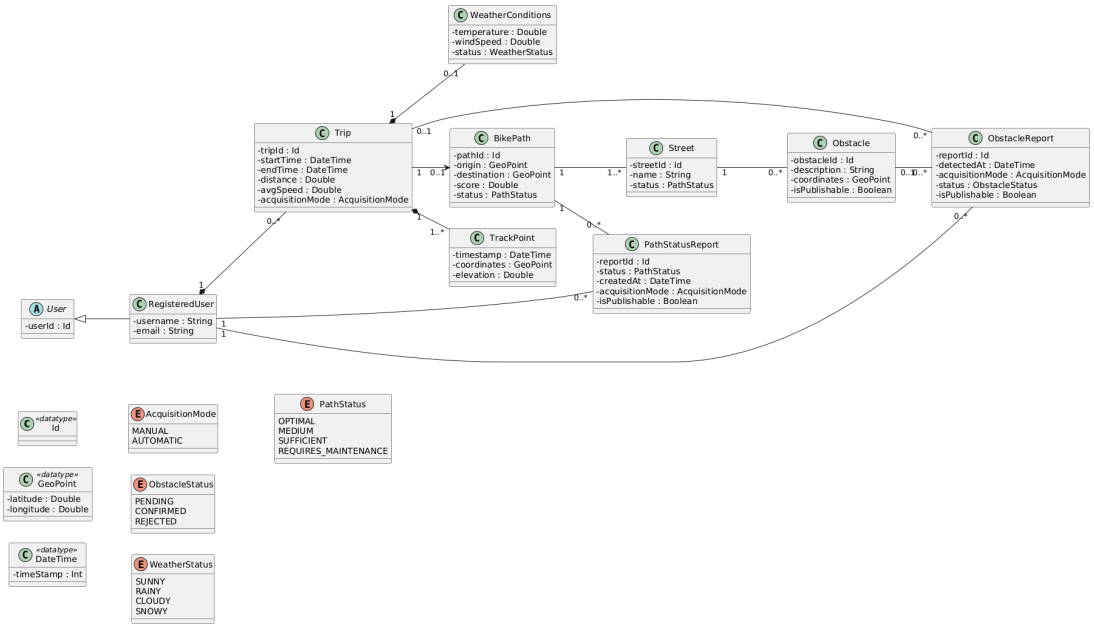


Figure 2.1: High Level View - Class Diagram

The class diagram in Figure 2.1 presents the conceptual domain model of the *Best Bike Paths* system. It focuses on the main entities and relationships that characterize cycling activities, reported obstacles, path conditions, and contextual information. This diagram does not describe how the system is implemented. Architectural aspects, technical components, and service interfaces for data acquisition and processing will be introduced and detailed later in the Design Document (DD).

At the core of the model is the abstraction `User`, specialized by `RegisteredUser`. A `RegisteredUser` represents an authenticated user of the platform, identified by a unique `userId` and characterized by basic account information such as `username` and `email`. Each registered user can own multiple `Trip` instances, corresponding to the cycling activities recorded over time.

A `Trip` describes a single cycling session. It stores a unique `tripId`, the `startTime` and `endTime`, the total `distance`, the `avgSpeed`, and an `acquisitionMode` attribute

indicating whether the trip was collected manually or automatically. A trip is decomposed into a sequence of `TrackPoint` objects, related to the trip through a composition. Each `TrackPoint` carries a `timestamp`, a geographic position (`coordinates` of type `GeoPoint`), and an `elevation`, capturing the detailed trace followed by the cyclist.

Trips can optionally be associated with a `BikePath`, which represents a meaningful route between two `GeoPoint` locations (`origin` and `destination`). A `BikePath` maintains an overall `score` and a `status` of type `PathStatus`, reflecting its current quality (for example optimal, medium, sufficient, or requiring maintenance). Each bike path is composed of one or more `Street` objects. A `Street` is identified by a `streetId` and a `name`, and it also holds a `status` of type `PathStatus` to represent the condition of that specific segment of the path.

The model explicitly represents user feedback on path quality through the `PathStatusReport` class. A `PathStatusReport` describes a single contribution provided by a user about the condition of a given `BikePath`. It includes a unique `reportId`, the reported `status` (`PathStatus`), the `createdAt` timestamp, and the `acquisitionMode` that distinguishes between manually entered and automatically derived feedback. The `isPublishable` flag indicates whether this piece of information can be made available to other users. Each `PathStatusReport` is associated with exactly one `RegisteredUser` and one `BikePath`, and the collection of reports contributes to the aggregated score and status maintained by the path.

Obstacles along the streets are modeled through the `Obstacle` and `ObstacleReport` classes. An `Obstacle` represents a consolidated unsafe element on the infrastructure, such as a pothole or an irregular surface. It has a unique `obstacleId`, a textual `description`, a position given by `coordinates`, and an `isPublishable` attribute specifying whether the obstacle is visible to the community. Obstacles are linked to `Street` objects, indicating on which street they are located. The life cycle and validation of obstacles are represented by `ObstacleReport`. Each `ObstacleReport` records a detection event with a unique `reportId`, the `detectedAt` time, the `acquisitionMode` (manual or automatic), and a `status` of type `ObstacleStatus`, which can be pending, confirmed, or rejected. The `isPublishable` attribute mirrors whether that specific report can be shared. Reports are created by `RegisteredUser` instances, may originate during a `Trip`, and can optionally be associated with a consolidated `Obstacle`. This allows the system to represent both preliminary detections and validated obstacles confirmed by multiple users.

Environmental context for each trip is captured by the `WeatherConditions` value object. A `Trip` may own at most one `WeatherConditions` instance, modeled as a composition.

This class stores temperature, wind speed, and a `status` of type `WeatherStatus` (such as sunny, rainy, cloudy, or snowy), thus enriching the activity with information about the weather at the time it was performed.

The diagram also includes simple data types and enumerations used across the domain. Identifiers are abstracted as `Id`, spatial positions as `GeoPoint`, and timestamps as `DateTime`. The `AcquisitionMode` enumeration distinguishes manual and automatic data acquisition. `PathStatus`, `ObstacleStatus`, and `WeatherStatus` enumerate the possible qualitative states for paths, obstacles, and weather respectively.

Overall, this class diagram describes the problem domain relevant to *Best Bike Paths*. It shows how user activities, routes, streets, obstacles, feedback on path quality, and contextual weather conditions are structured and interrelated at a conceptual level. It does not prescribe how these entities are stored, accessed, or computed at runtime. Such architectural and implementation details, including components for sensor management, trip recording, external map or weather services, and their interfaces, are intentionally omitted from the RASD domain model and will be addressed in the Design Document.

Clarifications on Domain Modeling Consistency

The class diagram in Figure 2.1 adopts a conceptual modeling approach that intentionally abstracts from implementation and persistence details. Some design choices are clarified below to ensure full consistency between the textual description and the UML model.

Trips are modeled as being owned by a `RegisteredUser` through a composition relationship. This implies a strong lifecycle dependency: a `Trip` cannot exist independently of its owning user and is removed if the corresponding account is deleted. While this dependency is not always made explicit in the scenarios, it is assumed throughout the domain model.

The notion of publication for trips and bike paths is treated as a process-level concern rather than a structural property of the domain entities. Consequently, the `BikePath` class does not include explicit attributes such as `isPublishable` or `publishedAt`. A bike path is considered publicly available when it exists in the shared dataset and is supported by publishable user contributions, such as validated `PathStatusReport` instances. The rules governing visibility and publication are enforced by application logic and are therefore intentionally excluded from the domain model.

Environmental information is modeled at the level of individual trips through the `WeatherConditions` value object, which is optionally composed by a `Trip`. This reflects the assumption that

meteorological data is contextual to the time and location of a specific cycling activity and is not aggregated or shared across multiple trips or bike paths.

Although the system allows both registered and unregistered cyclists to interact with the application, only registered users are represented as persistent entities in the domain model. Unregistered cyclists are treated as transient actors who can access and visualize aggregated information without being explicitly modeled as domain objects.

These clarifications ensure that the textual description, scenarios, and functional requirements remain fully aligned with the conceptual structure expressed by the UML class diagram, while preserving a clear separation between domain modeling and architectural or implementation concerns.

2.2. Product Functions

The following section describes the main functions of the system.

- **Account creation**:

  The account creation process allows a visitor to register as a BBP user through a structured and controlled workflow involving both user-side interactions and back-end validations. The process starts when the visitor opens the sign-in page and submits a proposed email address. The BBP backend first checks whether the email is already associated with an existing account. If the user is already registered, the process is aborted and an appropriate notice is issued.

  If the email is not registered, the user is asked to insert and confirm a password. This step is governed by a subprocess that verifies compliance with security requirements and checks that the two password entries correspond. In case of non-compliance or mismatch, error messages are returned and the user is allowed to retry, with the number of attempts being monitored. If the maximum number of attempts is exceeded, the process is aborted.

  Once valid credentials are provided, the backend registers the user in a provisional state and sends a confirmation email containing a verification code. The user must then insert the received confirmation code within a defined time window. This verification phase includes multiple subprocesses handling incorrect codes, timeouts, and email resend requests. The system limits both the number of code insertion attempts and the number of resend requests. Exceeding these limits results in a timeout or process termination with an explicit notice.

If the confirmation code is correct and received within the allowed constraints, the registration is finalized and the user account is activated. Upon successful confirmation, the user can immediately start editing their profile. The completion of this process grants access to all core BBP functionalities, including recording and storing personal cycling trips, viewing statistics, and actively contributing to the platform by publishing information on bike path conditions and obstacles.

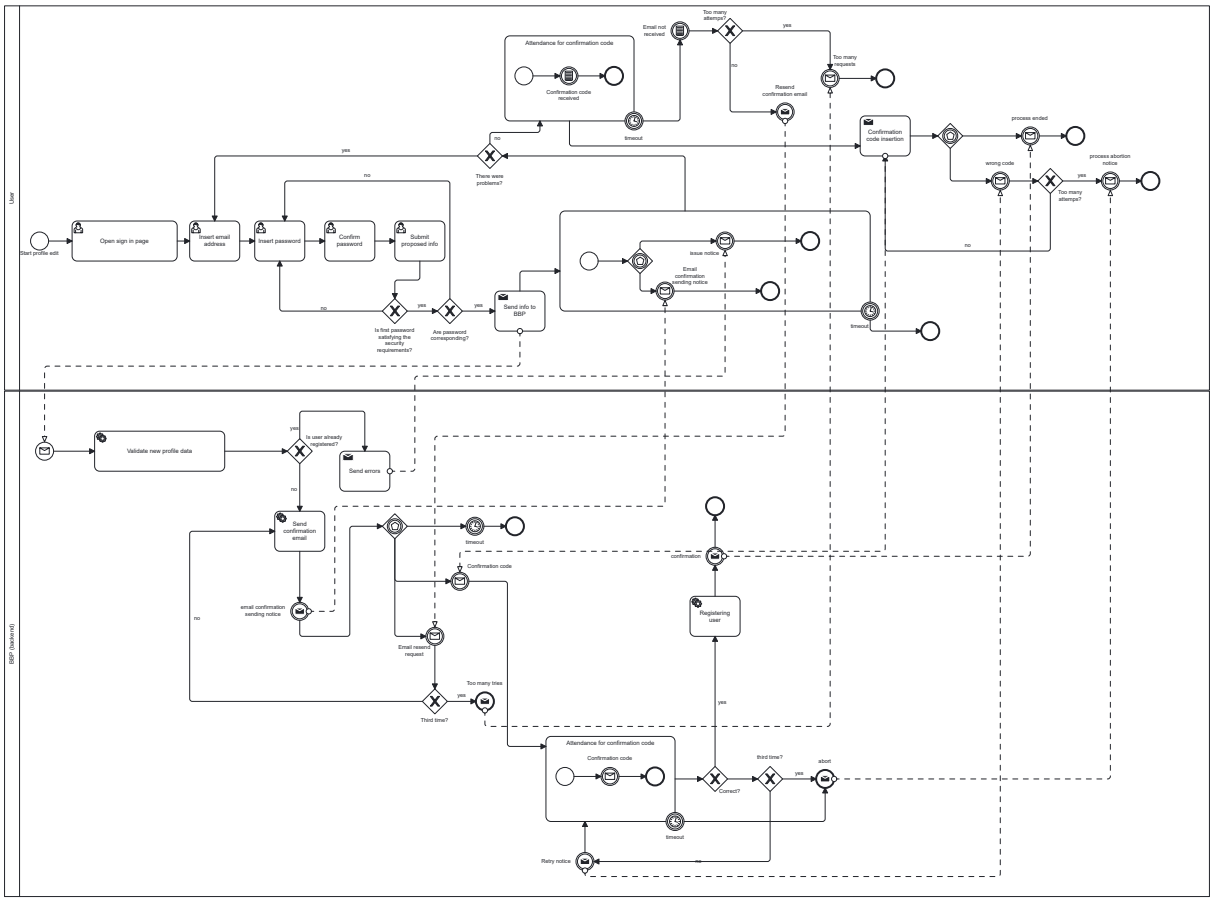


Figure 2.2: Account Creation - Product Function

- **Edit Profile**:

  The Edit Profile functionality enables authenticated users to update and maintain their personal information within the Best Bike Paths system through a controlled interaction with the backend services. The process starts when the user accesses their profile page and explicitly opens the edit mode. At this stage, the system exposes editable profile fields, excluding the email address, which is managed through a separate confirmation-driven process.

  The user can modify personal attributes such as display preferences or other non-

sensitive identification data. Once the changes are submitted, the BBP backend executes a validation subprocess to verify data consistency, format correctness, and compliance with system constraints. If validation errors are detected, the system returns detailed feedback, allowing the user to correct the input and resubmit the data.

If the provided information is valid, the backend updates the user profile in the database and sends a confirmation notice to the user, signaling the successful completion of the operation. The process also accounts for exceptional situations, such as inactivity timeouts, in which case the edit session is terminated and an explicit notice is issued. Upon successful update, all subsequent interactions with the platform reflect the newly stored profile information.

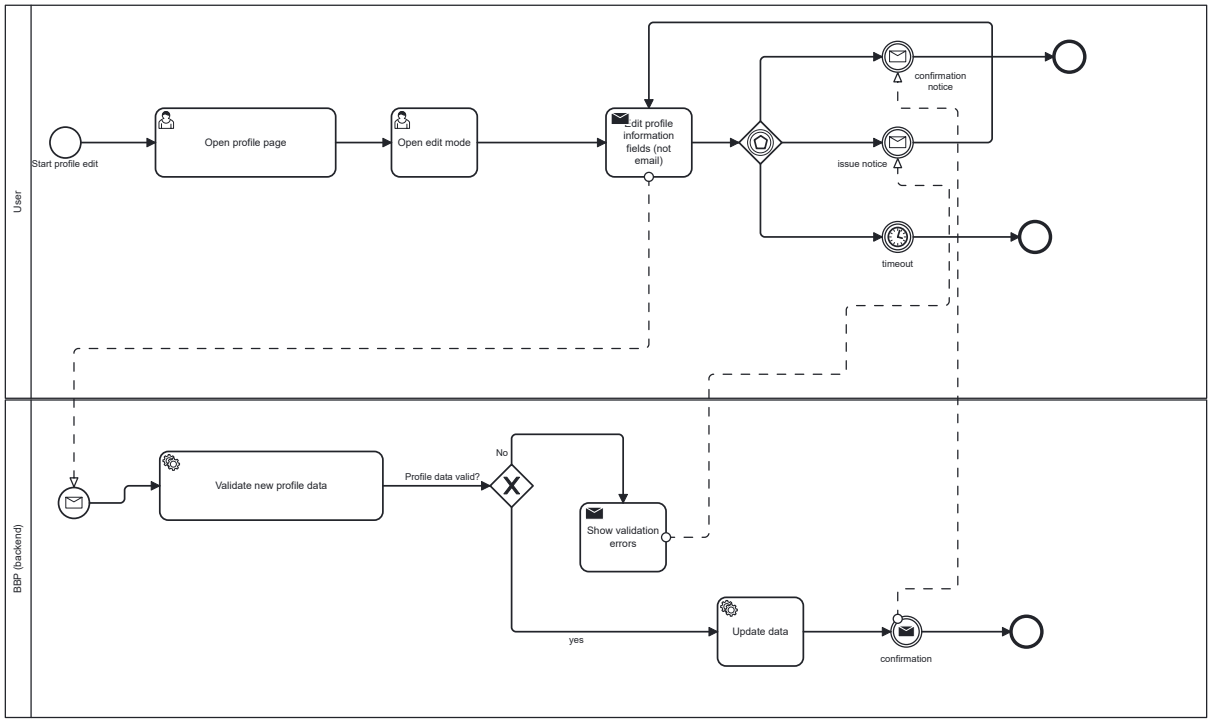


Figure 2.3: Edit profile - Product Function

- **Publish Trip**: Publish Trip is the functionality that allows a registered user to make one of their previously saved cycling trips publicly available, transforming it from a private draft into a shared resource within the Best Bike Paths (BBP) platform. This phase represents a critical transition in the lifecycle of a trip, as additional validation, enrichment, and evaluation processes are triggered exclusively at publication time.

  The process starts when the user accesses the saved trips section and selects a specific

trip to publish. After confirming the request and providing the required information, the system forwards the publish request to the BBP backend for processing.

Upon receiving the request, the backend initiates a subprocess aimed at improving data quality and consistency. This subprocess searches for other published trips that share one or more streets with the selected trip. If no common streets are found, the trip can be published directly.

If common streets are detected, the backend compares the street-related information contained in the newly published trip with existing data derived from other users' trips. A decision subprocess evaluates whether discrepancies or differences are present. When differences are identified, the system determines which information should be considered authoritative and updates the status of the affected streets, potentially marking them as problematic. These updates are then published to the shared dataset. This logic adheres to the system's requirement to merge information based on freshness and the number of confirming data points.

During this enrichment and validation phase, the system evaluates the quality of the trip and assigns a trip score. This score is computed exclusively at publication time, as it depends on the comparison with existing trips and on the consistency and reliability of the associated street data. Prior to publication, saved trips do not have an assigned score.

The update process may extend further if additional streets are discovered through other related trips, ensuring that all impacted data remains consistent. If errors occur at any stage of the publishing, comparison, or evaluation process, the procedure is terminated and an error notification is sent to the user.

When the publishing process completes successfully, the user receives a confirmation message. In cases where extended analysis or propagation of updates is required, the user is informed that the trip has been successfully published, but that some updates may take additional time to fully propagate.

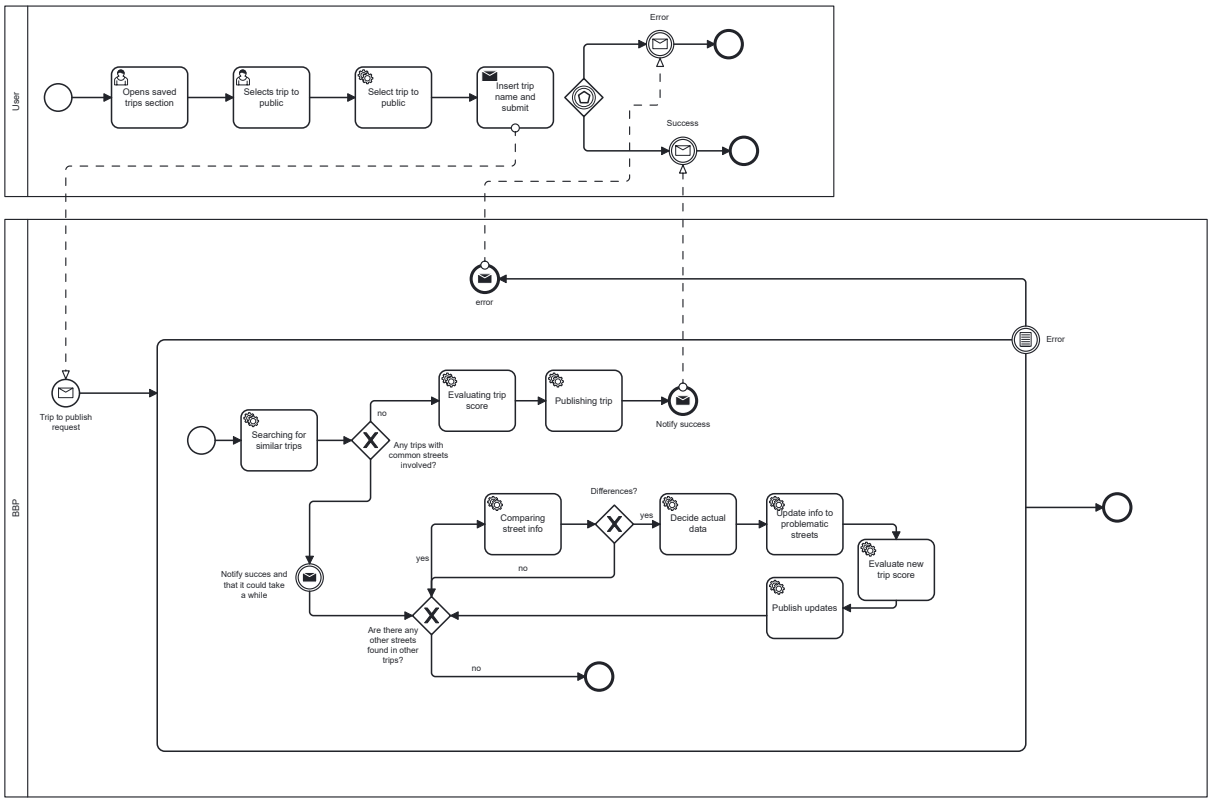


Figure 2.4: Publish Trip - Product Function

- **Automatic Trip**:

  The Automatic Trip functionality enables the system to record a user's cycling activity in real-time by leveraging the mobile device's onboard sensors, such as the gyroscope, accelerometer, and GPS. The process is initiated when the user opens the recording page and triggers the start command. The system immediately performs a preliminary check to verify if the application has the necessary permissions to access the required sensors. If permissions are missing, the user is prompted to grant them; if the user denies permission, the process cannot proceed.

  Once permissions are secured, the system performs a functional check to ensure data is actually being received from the sensors. If no data is detected, the system reports an error and halts the operation. Upon successful validation of sensor availability, the system calibrates the sensors and configures the sampling rate. The recording phase then enters a continuous loop where, for every active sensor within its specific sample interval, the system reads, processes, and aggregates new data. This real-time data is visualized on the user's dashboard until the user explicitly issues a stop recording signal.

After the recording is stopped, the user reviews the trip details and confirms the session. The system attempts to enrich the recorded data with local weather information, if available, before sending the draft data to the BBP backend.

On the backend, the incoming data undergoes a coherence validation process to ensure the integrity of the trip trajectory and sensor metrics. If the data is valid, the trip is saved to the database, and a success notice is sent back to the user. If the data shows inconsistencies, the backend attempts to build suggestions to correct the path before notifying the user. Finally, the client application receives the response and, based on the validity of the street info returned, displays a success visualization to confirm the trip has been stored.

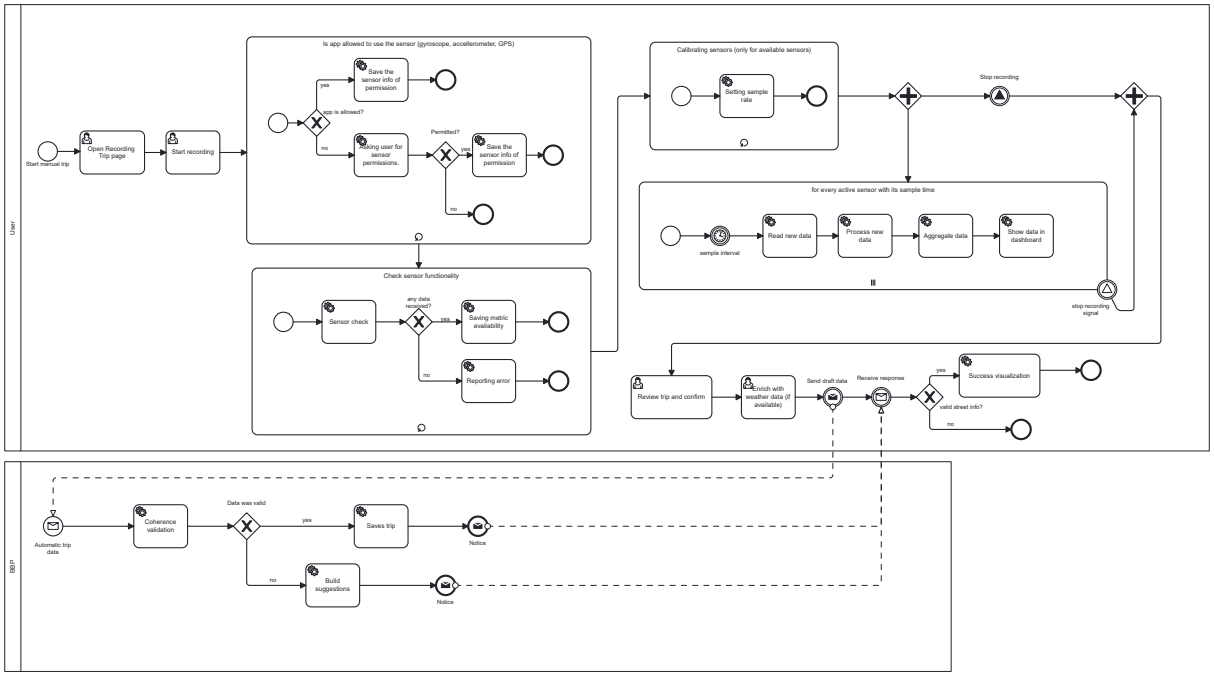


Figure 2.5: Automatic Trip - Product Function

- **Manual Trip and Edit Trip**:

  The Manual Trip and Edit Trip functionalities provide users with the flexibility to document cycling activities retrospectively or modify existing records without relying on real-time sensor recording. This process begins when the user navigates to their personal trips page. At this stage, a decision gateway determines the workflow: the user can either select the option to "create new manual trip" or choose a specific "personal trip to edit".

  The core of the trip construction happens through an iterative interaction loop

between the user, the BBP backend, and the external OpenStreetMap service. If the user wishes to add a street, they initiate a street search. This request is forwarded by the BBP system to OpenStreetMap, which processes the query and returns the relevant elaboration. The BBP backend then creates an appropriate data object for the street and presents the result to the user. The user then defines the specific path segments and assigns the relevant status to the street. Conversely, if the user needs to correct the route, they can select an already inserted street and delete it.

Once the user determines that the trip composition is complete, they proceed to review the trip details and confirm the submission. Before transmission, the system attempts to enrich the trip information with historical weather data, if available. The draft data is then sent to the backend for final processing.

Upon receiving the finished draft, the BBP system performs a coherence validation to ensure the logical continuity and data integrity of the route. If the data is valid, the system saves the trip and sends a success notice. If the validation fails, the system builds suggestions to rectify the errors and notifies the user, who must then resolve the validity issues before the trip can be successfully stored.

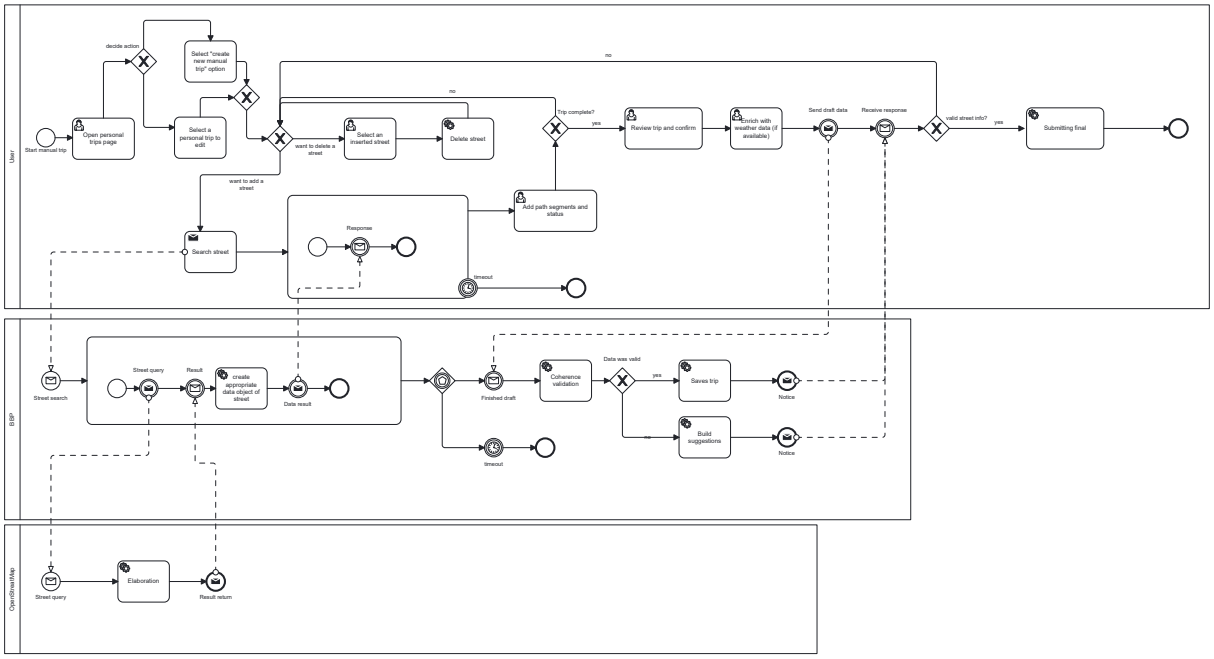


Figure 2.6: Manual Trip and Edit Trip - Product Function

- **Search Trip**:

  The Search Trip functionality empowers users to discover existing cycling routes within the BBP database based on specific geographical and qualitative criteria. The

process begins when the user accesses the search page and defines the parameters for the desired trip. This involves inserting a start position and a final position, choosing the desired distance of the trip, and selecting filter conditions based on the trip score.

Once the criteria are defined and the request is submitted, the BBP backend processes the input by composing and executing a database query. The system then verifies if any data matches the user's search parameters. If no matching trips are found, a negative response is returned to the user. Conversely, if data is available, the backend formats the database results and sends a positive response containing the trip details.

On the client side, the application waits for the server's response, with a timeout mechanism in place to handle potential communication failures. Upon receiving the data, the application formats it for visualization, presenting the results to the user. At this stage, the user is offered the option to sort the results. If the user chooses to sort, they can select a specific sort parameter, prompting the system to re-render the data visualization accordingly. The process concludes when the user is satisfied with the view or chooses not to apply further sorting.

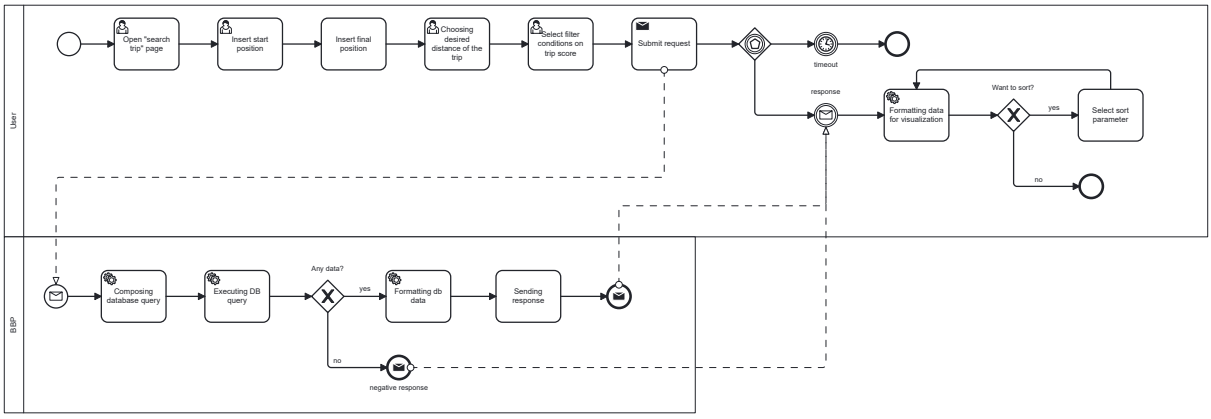


Figure 2.7: Search Trip - Product Function

- **Weather Conditions Retrieval (Subprocess)**:

  In several BPMN diagrams related to trip recording and submission, the enrichment of a trip with weather information is represented as a single atomic activity for readability and space constraints. In practice, this activity corresponds to a structured client-side subprocess that interacts with an external weather service.

  The subprocess is triggered once a trip recording is completed and before the trip

data is sent to the BBP backend. The client application extracts the relevant contextual parameters from the recorded trip, including the geographic position, the date, and the time interval of the activity. These parameters are used to build a request addressed to the external weather service.

The weather service processes the request and attempts to return meteorological information corresponding to the specified location and time, such as temperature, wind conditions, and a qualitative weather status. Upon receiving the response, the client validates its correctness and completeness. If the response is valid, the trip data is enriched with the retrieved weather information and the subprocess terminates successfully.

If the request fails due to a timeout or if the response is invalid or unavailable, the subprocess terminates without enriching the trip. In this case, the absence of weather data does not block the overall trip submission process, which continues with the available information. This design choice ensures robustness against temporary unavailability of external services and maintains a smooth user experience.

Figure 2.8 details the internal flow of this subprocess, which is abstracted as a single task in higher-level BPMN diagrams to preserve clarity.

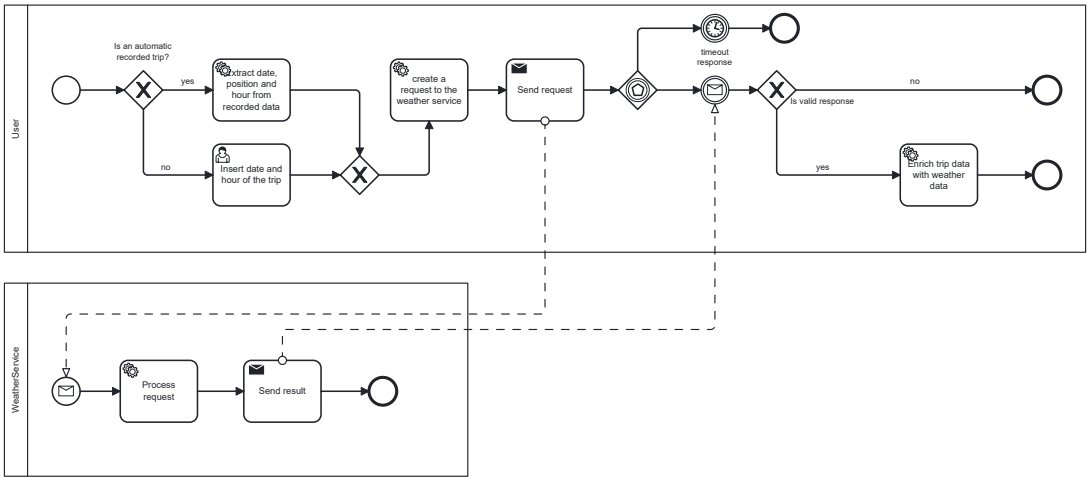


Figure 2.8: Weather Conditions Retrieval - Client-side Subprocess

2.3. User Characteristics

The Best Bike Paths (BBP) system is designed around a single primary actor category: cyclists. These users directly interact with the system by providing information about their trips, obstacles, and the condition of bike paths. Although the platform relies on external map and weather services, these components do not constitute actors, since they do not actively interact with the system as users. Therefore, cyclists represent the only true user-facing category in the domain.

1. **Cyclists (CYs)**: individuals who use the BBP platform while cycling.
   (a) Cyclists may access the system anonymously or through a registered account.
   (b) *Registered cyclists* can record trips, provide feedback on path conditions, report obstacles, and validate automatically detected obstacles.
   (c) They may choose whether their reports or feedback become publishable to the community.
   (d) *Unregistered cyclists* may only browse aggregated information about paths, obstacles, and cycling conditions, without contributing data.
   (e) In both cases, cyclists benefit from enriched information about bike routes, including street conditions and past observations collected from the community.

All additional components (e.g., map providers, weather services, sensors) are external technical dependencies and will be discussed separately in the Design Document.

2.4. Assumptions, Dependencies and Constraints

2.4.1. Assumptions

Assumption	Description
A01	Cyclists use a smartphone capable of running the BBP application and supporting the required sensing features.
A02	Device location services (GPS) are enabled when using automatic acquisition mode, allowing the system to reconstruct trips reliably.
A03	Smartphones include functioning motion sensors (such as accelerometers) that can detect vibrations and help identify potential obstacles.

A04	Cyclists have a stable internet connection during or after their trips to upload data, synchronize reports, and retrieve updated information.
A05	Users submitting manual path status reports or obstacle reports provide information that reasonably reflects the true conditions of the streets.
A06	External weather and map providers return accurate, timely, and consistent information used to enrich trip data and support path evaluation.
A07	The mobile device clock is set correctly, allowing timestamps to be meaningful for trip reconstruction and report creation.

Table 2.1: Domain Assumptions Table

2.4.2. Dependencies

Dependency	Description
D01	The system depends on external map services to retrieve geographic data, validate street names, compute paths, and represent routes.
D02	The system depends on external weather providers to retrieve real-time or historical weather conditions associated with trips.
D03	The mobile operating system must grant the application access to device sensors such as GPS, accelerometers, and motion detectors.
D04	The application requires stable internet connectivity for synchronizing trips, uploading reports, downloading path conditions, and retrieving external data.
D05	Mobile platform APIs must function correctly to access background location updates during trips.

Table 2.2: Domain Dependencies Table

2.4.3. Constraints

Constraint	Description
C01	The system must comply with data protection and privacy regulations concerning the collection, processing, and storage of cyclists' location data.
C02	BBP must operate on mobile devices with heterogeneous hardware capabilities, meaning data quality may vary across devices.

C03	The system is constrained by sensor precision: inaccurate GPS, unstable accelerometer readings, or motion sensor noise may impact trip reconstruction and obstacle detection.
C04	Only publishable and validated data (e.g., confirmed obstacles and user-approved reports) can be displayed to the community.
C05	The platform should remain functional even with intermittent connectivity, but certain operations (such as uploading data or fetching updated path statuses) are limited without internet access.
C06	Device battery consumption must be considered, as continuous GPS and sensor usage can reduce battery life during long trips.

Table 2.3: Domain Constraints Table

3 | Specific Requirements

This chapter describes the specific requirements of the Best Bike Paths (BBP) system. It provides detailed information on the external interfaces through which users and external services interact with the system. These requirements supplement the functional and domain descriptions by clarifying how BBP is accessed, which devices it relies on, and how it communicates with external data sources.

3.1. External Interface Requirements

3.1.1. User Interfaces

The primary user interface of BBP is designed with a mobile-first approach, as cyclists typically interact with the system through smartphones while biking or during breaks. The interface allows registered users to access core functionalities, including:

- starting, pausing, and stopping the recording of a trip to track cycling activities;
- visualizing the reconstructed path, associated statistics (e.g., speed, distance), and enriched meteorological information;
- submitting manual data regarding path status and obstacles;
- confirming or correcting automatically detected information (e.g., potholes) to prevent false positives before publication;
- browsing existing bike paths between a specific origin and destination, visualized based on their calculated score;
- accessing previously recorded trips and personal contributions.

The user interface focuses on clarity and ease of interaction. While a web interface may be provided for browsing statistics on larger screens, the automated data acquisition features rely on a mobile application environment.

3.1.2. Hardware Interfaces

BBP relies heavily on smartphones, which act as the primary sensing and interaction devices. The system requires access to the following hardware components:

- **GPS and location sensors** to reconstruct the followed path and determine speed (used to infer biking activity);
- **Accelerometers and gyroscopes** to detect significant device movements and vibrations that suggest the presence of potholes or other obstacles;
- **Networking hardware** (Wi-Fi or mobile data) to allow synchronization of data and retrieval of external information;
- **Local device storage** to buffer trip data before upload.

3.1.3. Software Interfaces

BBP interacts with several external software services and platform-level interfaces necessary to provide complete and accurate information. These include:

- **Map service interfaces**, which provide geographic data and visualization capabilities for the paths;
- **Weather information providers**, which allow the system to enrich trip data with meteorological information (weather conditions, temperature, wind speed);
- **Mobile operating system APIs**, which grant access to device sensors such as GPS, accelerometers, and gyroscopes.

These external services are accessed programmatically by the BBP system. Specific technical details regarding protocols and data formats are deferred to the Design Document (DD).

3.1.4. Communication Interfaces

Communication between BBP components and external services relies on secure and widely adopted web protocols. Specifically:

- **HTTPS** is used for all communication between the client application and the backend server to ensure data security.
- **RESTful service endpoints** are used for structured data exchange between clients and the backend.

- **External API calls** are used to retrieve data from map and weather services.

No additional legacy communication channels are required by the system specification.

3.2. Functional Requirements

3.2.1. Requirements List

ID	Requirement
R01	The system shall allow cyclists to create a registered account using a valid email address.
R02	The system shall allow registered cyclists to log in and log out securely.
R03	The system shall allow unregistered cyclists to access publicly available information without authentication.
R04	The system shall allow registered cyclists to start a new trip recording session.
R05	The system shall record geographic coordinates at regular intervals during a trip.
R06	The system shall generate a sequence of `TrackPoint` objects for every recorded trip.
R07	The system shall compute summary statistics for each trip, including distance, duration, and average speed.
R08	The system shall allow cyclists to set the acquisition mode (manual or automatic) for a trip.
R09	The system shall associate weather conditions with a trip when external weather information is available.
R10	The system shall allow cyclists using manual acquisition mode to manually specify the streets forming their trip path.
R11	The system shall allow registered cyclists to submit obstacle reports during or after a trip.
R12	The system shall mark a newly submitted obstacle report as `PENDING`.
R13	The system shall allow cyclists to confirm or reject automatically detected obstacle reports.
R14	The system shall update the status of an obstacle report to `CONFIRMED` or `REJECTED` after user validation.
R15	The system shall associate confirmed obstacle reports with an existing obstacle or create a new `Obstacle`.

R16	The system shall allow cyclists to choose whether an obstacle report is publishable.
R17	The system shall store each obstacle together with its description and geographic coordinates.
R18	The system shall associate obstacles with the street on which they are located.
R19	The system shall update the visibility of an obstacle depending on the cyclist's publishability settings.
R20	The system shall represent bike paths as sequences of streets between an origin and a destination.
R21	The system shall display the current status and score of each bike path.
R22	The system shall allow registered cyclists to submit `PathStatusReport` entries for any bike path.
R23	The system shall allow cyclists to specify whether their path status report is publishable.
R24	The system shall update the aggregated status or score of a bike path based on submitted path status reports.
R25	The system shall allow cyclists to browse all publicly available bike paths.
R26	The system shall display the current condition of streets, including reported obstacles.
R27	The system shall allow cyclists to browse their previously recorded trips.
R28	The system shall allow cyclists to view details of each trip, including weather, track points, and obstacle reports.
R29	The system shall allow cyclists to mark any trip-related data (e.g., obstacle reports, path status reports) as publishable or private.
R30	The system shall ensure that private reports are not shown to other users.
R31	The system shall retrieve weather data from an external weather provider when available.
R32	The system shall retrieve map and geographic information from an external map provider.
R33	The system shall notify the user when sensors required for automatic acquisition mode are unavailable.
R34	The system shall notify the user if GPS signal quality is insufficient to record a trip.
R35	The system shall prevent the submission of incomplete or invalid obstacle reports.

Table 3.1: System Requirements Table

3.2.2. Use Case Diagrams

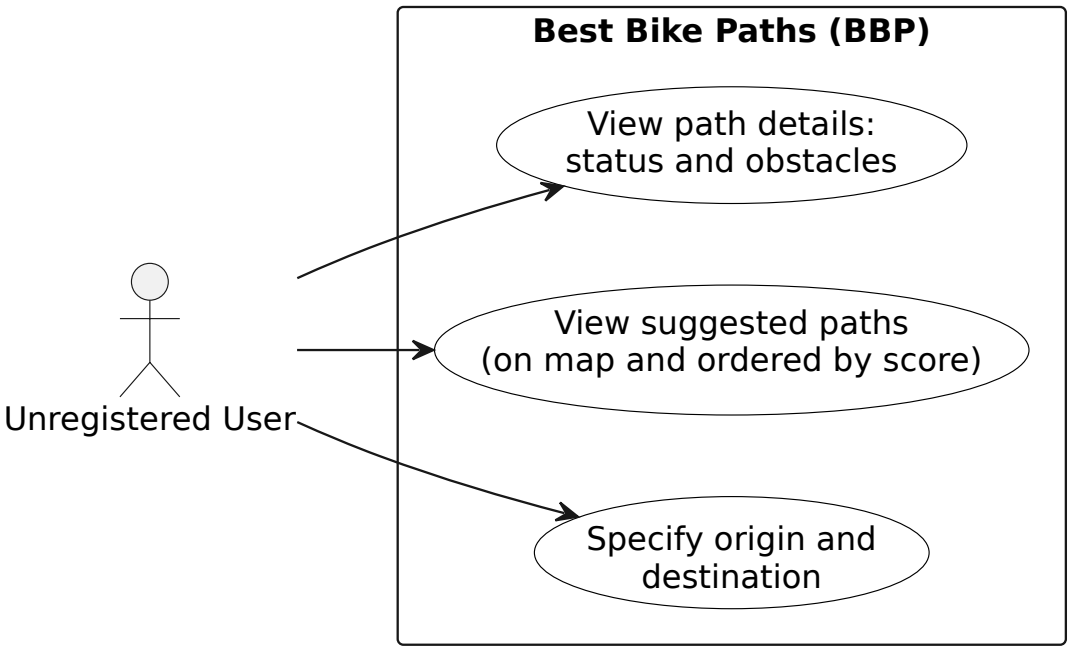


Figure 3.1: Unregistered User abilities - Use Case Diagram

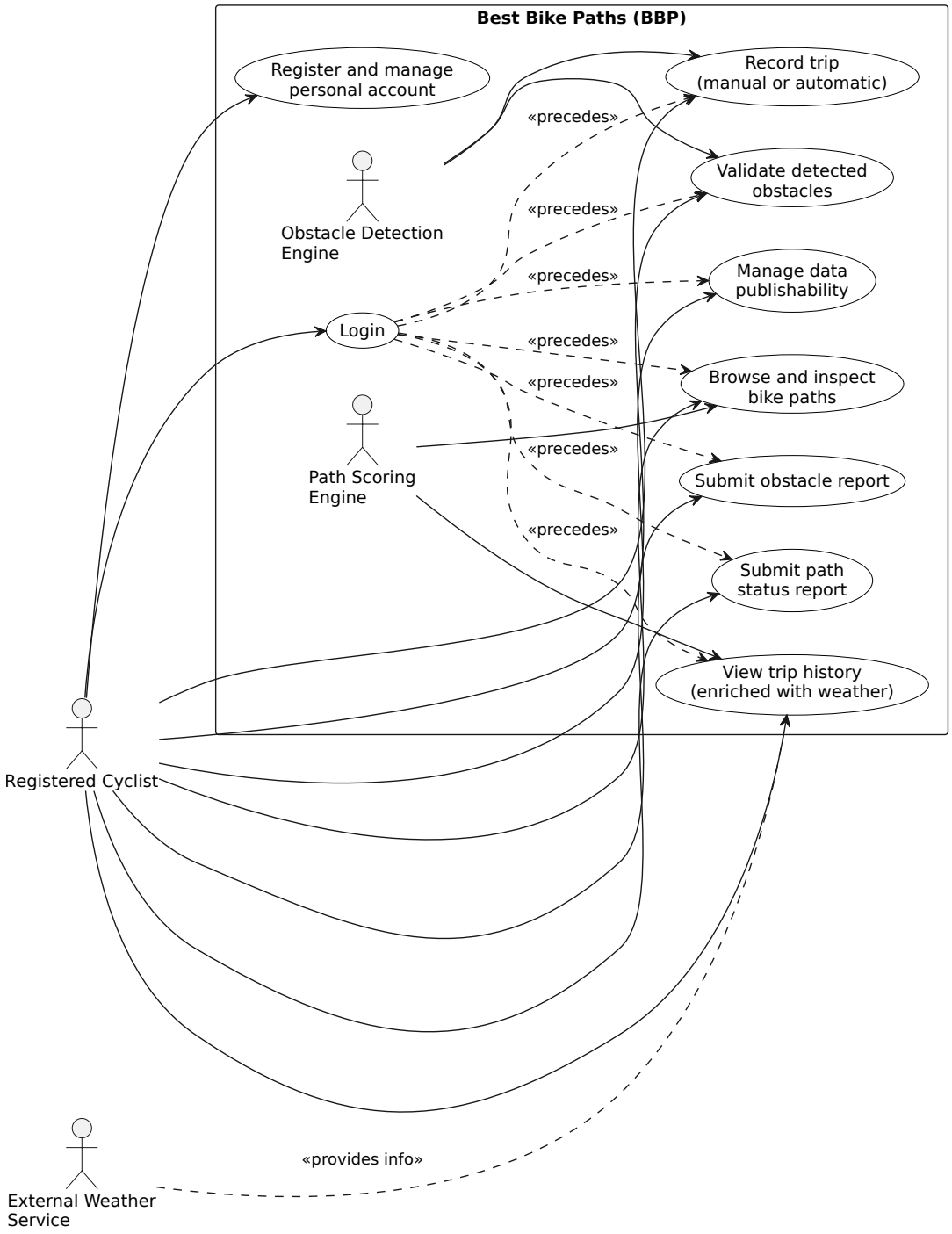


Figure 3.2: Registered Cyclist abilities - Use Case Diagram

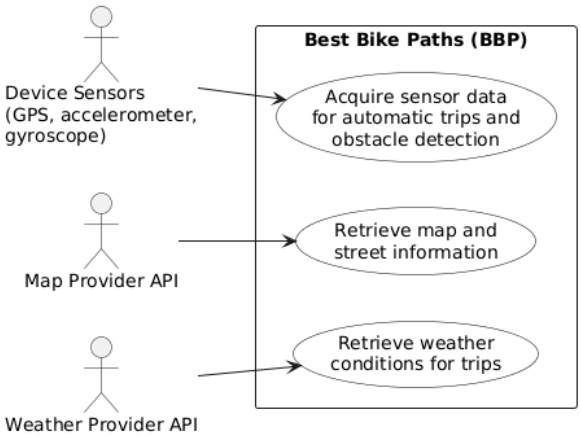


Figure 3.3: System and external services - Use Case Diagram

3.2.3. Use Cases

In this section, a detailed description of the main use cases of the system is presented. Each use case is described by actors, entry conditions, flow of events, exit conditions, and exceptions. The use case diagram shows the messages exchanged between the actors and the system and the functions called by them.

UC00: Cyclist registration

Actors:	Cyclist (unregistered user)
Entry Conditions:	• The cyclist has access to the BBP application or web interface. • The cyclist has a valid email address.

Flow of Events:	1. The cyclist opens the BBP application or web interface. 2. The cyclist selects the option *Register* or *Create account*. 3. BBP displays the registration form requesting at least email, username, and password. 4. The cyclist fills in the required fields and submits the form. 5. BBP forwards the registration data to the backend for validation. 6. The backend verifies: • that the email is not already associated with an existing account, • that all mandatory fields are present and valid (e.g., email format, password constraints). 7. If the data are valid and the email is not already registered, the backend creates a new `RegisteredUser` associated with the provided information. 8. The backend returns a success response to BBP. 9. BBP displays a confirmation message and may optionally redirect the cyclist to the login page.
Exit condition:	A new `RegisteredUser` account exists in the system and can be used to log in to BBP.
Exceptions:	1. The email is already registered: BBP informs the cyclist and suggests logging in or recovering the password. 2. One or more mandatory fields are missing or invalid: BBP highlights the errors and asks the cyclist to correct them. 3. The backend is temporarily unavailable or returns an error: BBP shows a generic error message and no account is created.

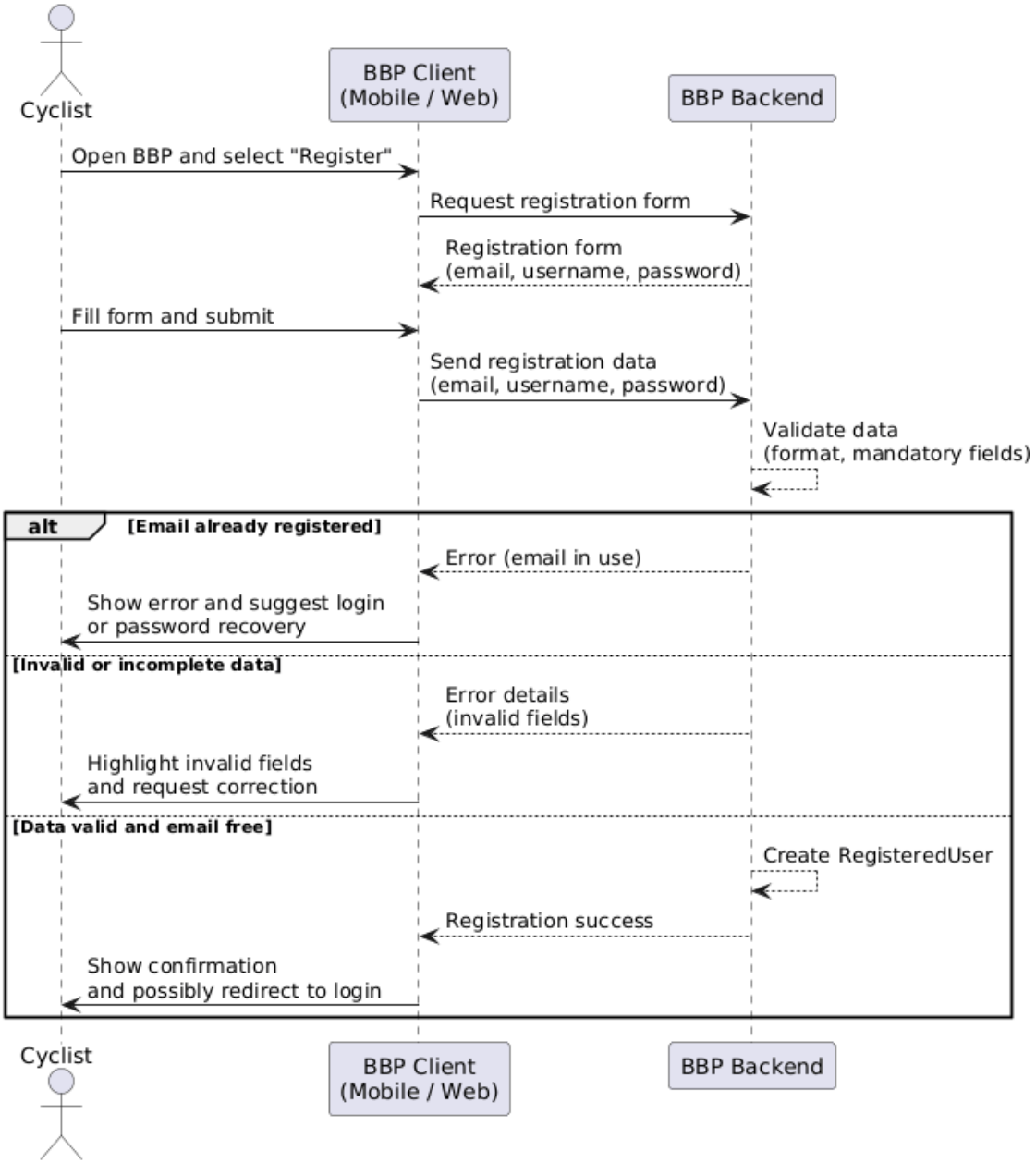


Figure 3.4: Cyclist registration - Sequence Diagram

UC01: Cyclist login

Actors:	Cyclist (registered user)
Entry Conditions:	• The cyclist has already created a BBP account. • The BBP application or web interface is reachable.
Flow of Events:	1. The cyclist opens the BBP application or web interface. 2. The cyclist selects the *Login* option. 3. BBP displays the login form requesting email and password. 4. The cyclist enters email and password and submits the form. 5. BBP receives the credentials and forwards them to the authentication service. 6. The authentication service validates the credentials. 7. If validation succeeds, BBP creates a session for the cyclist. 8. BBP redirects the cyclist to the main dashboard (e.g., trips overview or home screen).
Exit condition:	The cyclist is logged in and can access personal trips and contributions.
Exceptions:	1. The credentials are wrong: BBP shows an error message and asks the cyclist to retry. 2. The account is blocked or deleted: BBP denies access and informs the cyclist. 3. An internal error occurs in the authentication service: BBP shows a generic error message.

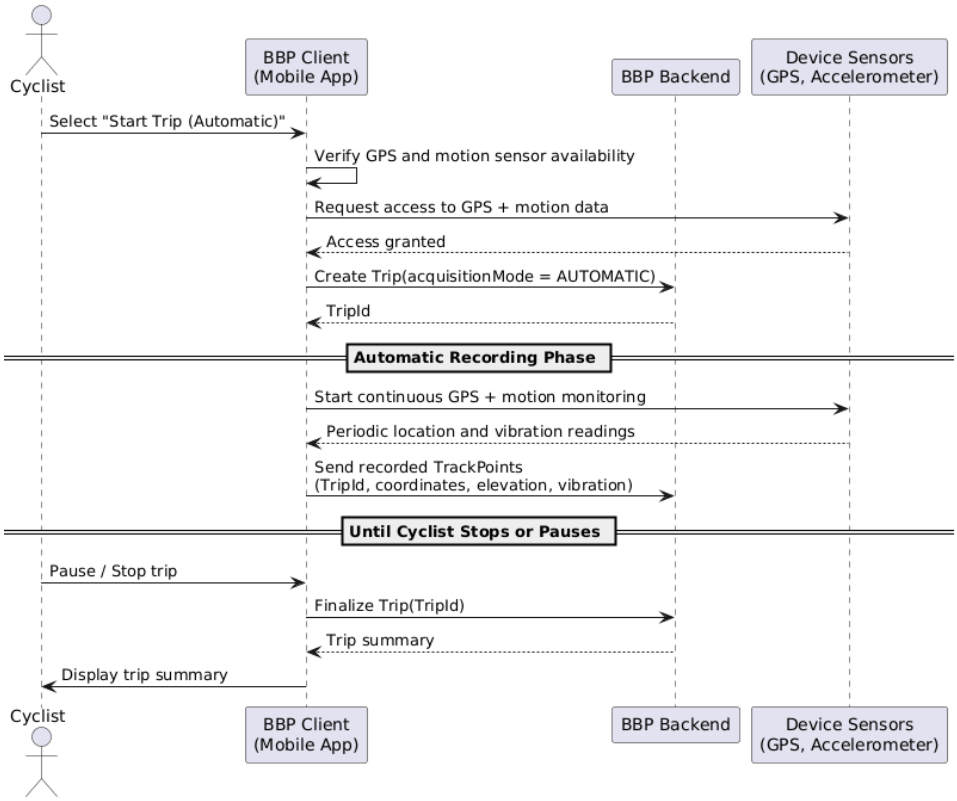


Figure 3.5: Cyclist Login - Sequence Diagram

UC02: Start automatic trip recording

Actors:	Cyclist (registered user)
Entry Conditions:	• The cyclist is authenticated in the BBP system. • The BBP mobile application is installed and running on a device with: – GPS and motion sensors (e.g., accelerometer, gyroscope), – granted permissions for location and motion access.

Flow of Events:	1. The cyclist opens the trip recording section of the application. 2. The cyclist selects the option *Start Trip (Automatic)*. 3. BBP verifies that GPS and motion sensors are available and that permissions are granted. 4. If the checks succeed, BBP sends a request to the backend to create a new `Trip` with `acquisitionMode = AUTOMATIC` associated with the cyclist. 5. The backend creates the Trip and returns the Trip identifier and initial metadata. 6. BBP starts reading location and motion data from the device sensors at regular intervals. 7. For each new reading, BBP derives speed and elevation (when available) and sends the data to the backend. 8. The backend stores each received point as a `TrackPoint` associated with the Trip. 9. The trip continues in active automatic recording until the cyclist explicitly stops the recording (handled by a separate use case) or closes the application.
Exit condition:	A Trip in automatic acquisition mode is active, and a sequence of `TrackPoint` objects is being stored for the cyclist.
Exceptions:	1. GPS or motion sensors are unavailable: BBP informs the cyclist that automatic recording cannot start and suggests enabling sensors or checking the device. 2. Required permissions are denied: BBP requests the necessary permissions; if the cyclist refuses, the trip is not started. 3. The backend is temporarily unavailable when creating the Trip: BBP shows an error message and does not start recording.

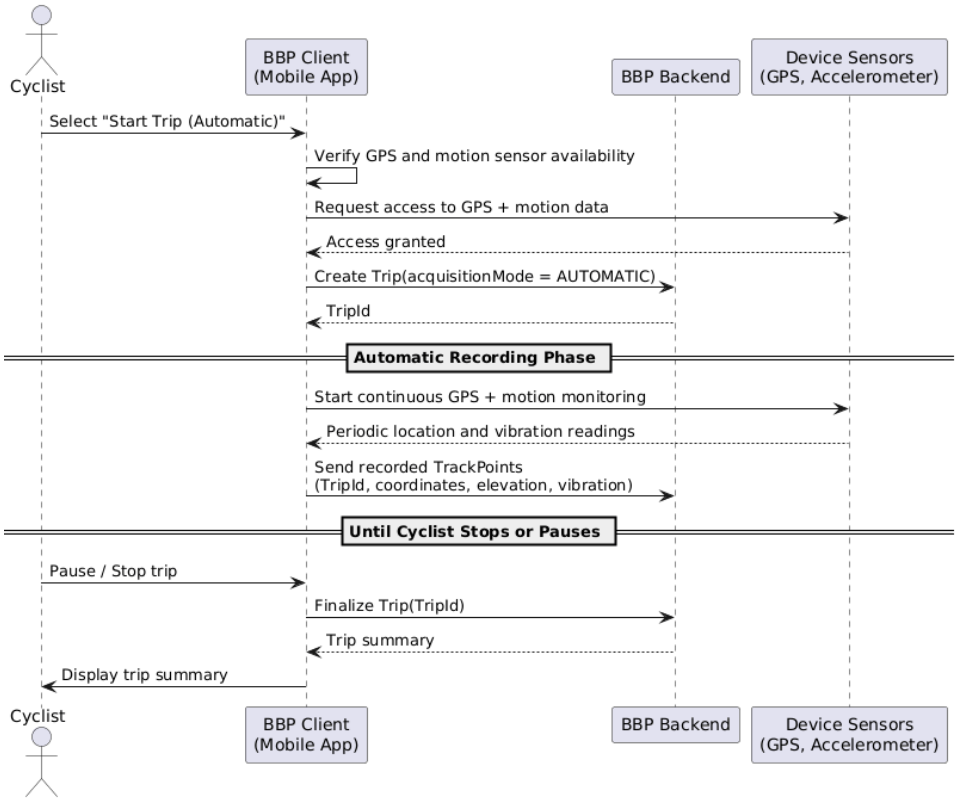


Figure 3.6: Start automatic trip recording - Sequence Diagram

UC03: Start manual trip recording

Actors:	Cyclist (registered user)
Entry Conditions:	• The cyclist is authenticated in the BBP system. • The BBP application or web interface is reachable.

Flow of Events:	1. The cyclist opens the trip recording section of the application. 2. The cyclist selects the option *Start Trip (Manual)*. 3. BBP displays an interface to specify the path manually (for example through a map or street search). 4. The cyclist selects or enters the streets that will compose the trip path. 5. BBP validates the selected streets using available map information. 6. BBP creates a new `Trip` with `acquisitionMode = MANUAL` associated with the cyclist. 7. BBP associates the selected streets with the trip (and with a corresponding `BikePath`, if applicable). 8. BBP marks the trip as started and ready to be controlled (pause, resume, stop) by the cyclist.
Exit condition:	A Trip in manual acquisition mode is active and associated with a set of streets.
Exceptions:	1. One or more street names or selections cannot be validated: BBP notifies the cyclist and requests correction. 2. The cyclist cancels the manual trip creation before confirming: no Trip is created. 3. An internal error occurs while creating the Trip: BBP shows an error message and suggests retrying.

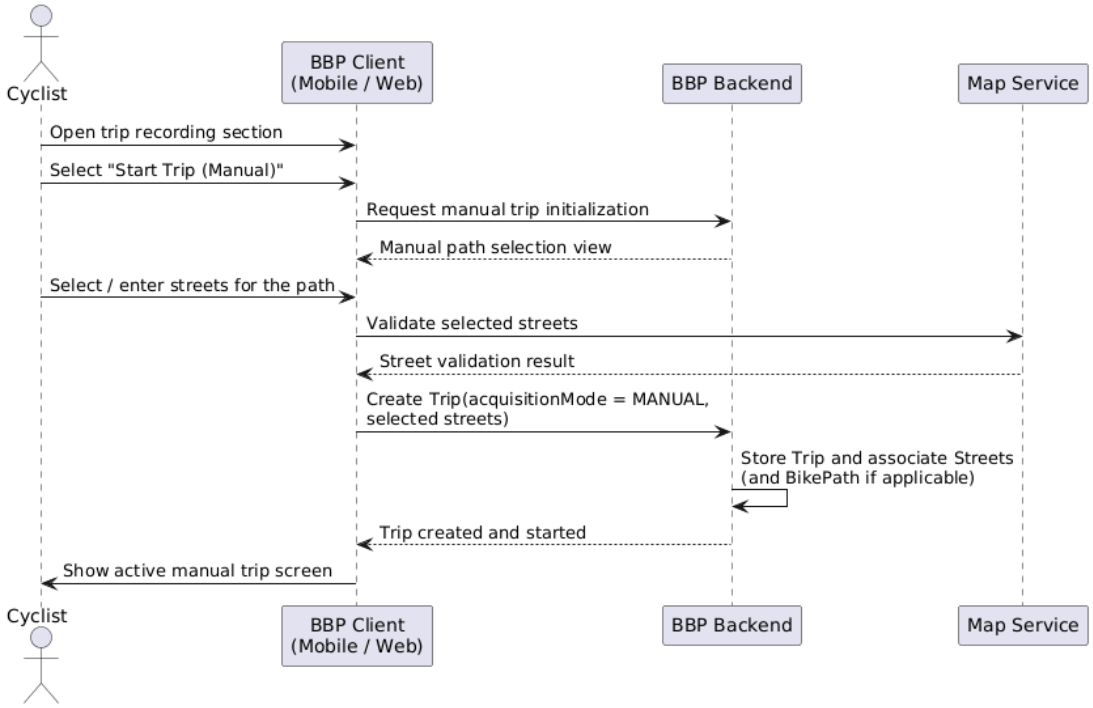


Figure 3.7: Start manual trip recording - Sequence Diagram

UC04: View trip history and details

Actors:	Cyclist (registered user)
Entry Conditions:	• The cyclist is authenticated in the BBP system. • At least one Trip is associated with the cyclist.

Flow of Events:	1. The cyclist navigates to the "My Trips" section of the application. 2. BBP requests the list of trips associated with the cyclist. 3. The backend retrieves and sends the list of Trips (with summaries such as date, duration, distance). 4. The cyclist selects one specific Trip. 5. BBP requests the detailed information of that Trip. 6. The backend returns the Trip details including: • track points (if automatically acquired), • associated BikePath (if any), • weather conditions (if recorded), • obstacle reports created during the trip. 7. BBP displays all retrieved information to the cyclist.
Exit condition:	The cyclist visualizes the selected Trip with all available details.
Exceptions:	1. No trips exist: BBP displays an empty history message. 2. Backend temporarily unavailable: BBP displays an error and offers retry. 3. Trip details incomplete (for example missing map data): BBP displays available parts and warns the user.

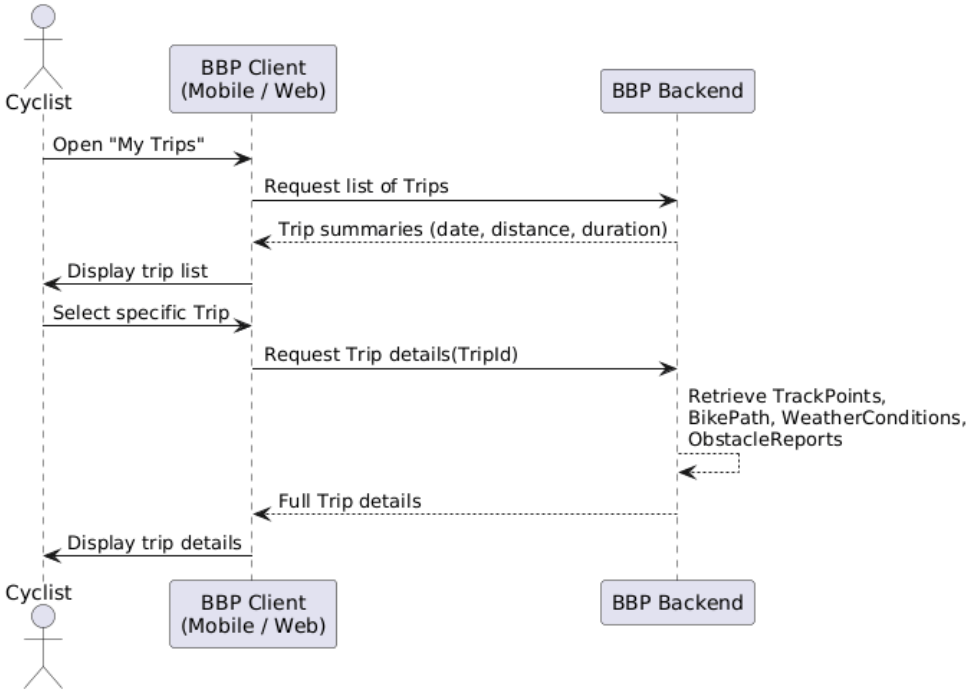


Figure 3.8: View trip history and trip details - Sequence Diagram

UC05: Submit obstacle report

Actors:	Cyclist (registered user)
Entry Conditions:	• The cyclist is authenticated in the BBP system. • The cyclist has identified a potential obstacle along a street or along a recorded trip.

Flow of Events:	1. The cyclist selects the *Report obstacle* option from the application. 2. BBP displays a form requesting the obstacle location (for example map position or current GPS position), a short description, and optional additional notes. 3. The cyclist provides the mandatory information and optionally adds extra details. 4. The cyclist confirms the submission of the obstacle report. 5. BBP creates a new `ObstacleReport` with: • the provided location and description, • the acquisition mode (manual or automatic, depending on the current trip), • status set to `PENDING`, • a link to the cyclist (RegisteredUser), • an association to the current Trip if the report is created during a trip. 6. BBP requests the cyclist to choose whether the report should be publishable to other users. 7. The cyclist specifies the publishability preference. 8. BBP stores the `ObstacleReport` together with the chosen `isPublishable` value.
Exit condition:	A new `ObstacleReport` with status `PENDING` is stored in the system and associated with the cyclist and the specified location.
Exceptions:	1. Mandatory information (such as location or description) is missing: BBP highlights the missing fields and does not create the report. 2. The map or location service is temporarily unavailable: BBP informs the cyclist and suggests retrying later. 3. An internal error occurs while saving the report: BBP shows an error message and the report is not stored.

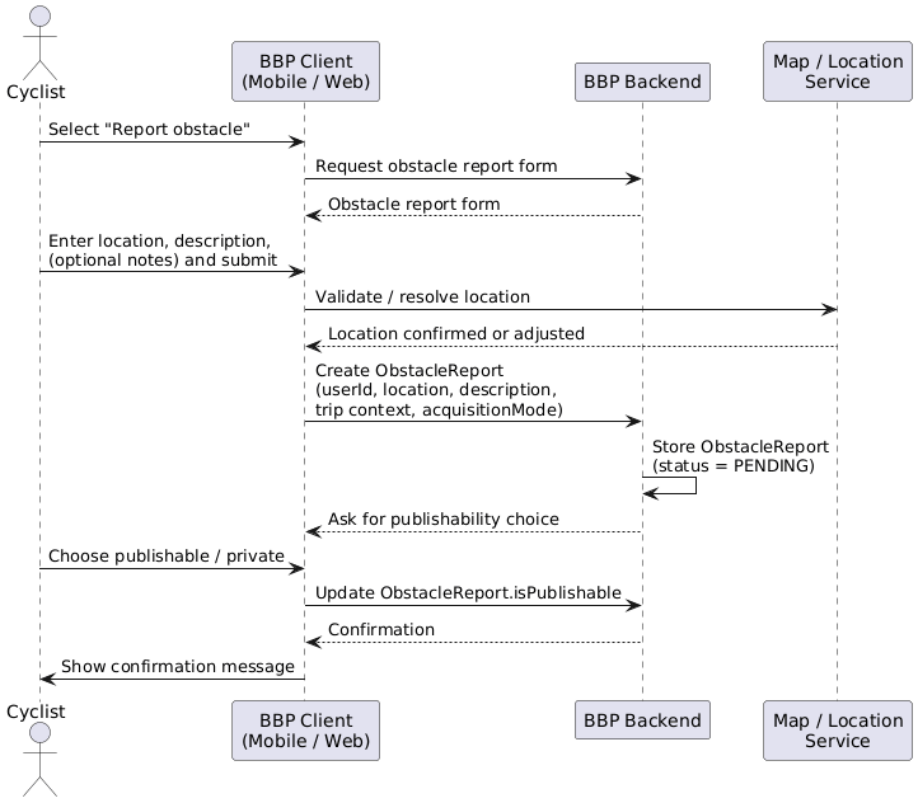


Figure 3.9: Submit obstacle report - Sequence Diagram

UC06: Validate detected obstacle

Actors:	Cyclist (registered user)
Entry Conditions:	• The cyclist is authenticated in the BBP system. • At least one `ObstacleReport` with status `PENDING` is available in the area of the cyclist or along a recorded trip.

Flow of Events:	1. During or after a trip, BBP notifies the cyclist about one or more potential obstacles (pending reports) near the current path or location. 2. The cyclist opens the obstacle notification or the obstacle list. 3. BBP displays details of the selected pending `ObstacleReport`, including location and description. 4. The cyclist inspects the area and chooses to either *Confirm* or *Reject* the obstacle. 5. The cyclist submits their choice. 6. BBP sends the updated decision to the backend. 7. The backend updates the `ObstacleReport.status` to: • `CONFIRMED` if the cyclist confirms the obstacle; • `REJECTED` if the cyclist rejects the obstacle. 8. If the report is confirmed and sufficient evidence exists (for example enough confirmed reports in the same area), the backend associates the report with an existing `Obstacle` or creates a new one. 9. BBP informs the cyclist that the obstacle report has been updated.
Exit condition:	The selected `ObstacleReport` has status `CONFIRMED` or `REJECTED`. Confirmed reports may contribute to creating or updating an `Obstacle`.
Exceptions:	1. The pending report is no longer available (for example already validated by others): BBP informs the cyclist and refreshes the list. 2. Backend temporarily unavailable: BBP displays an error message and suggests retrying later. 3. The cyclist closes the notification without making a choice: no change occurs to the report.

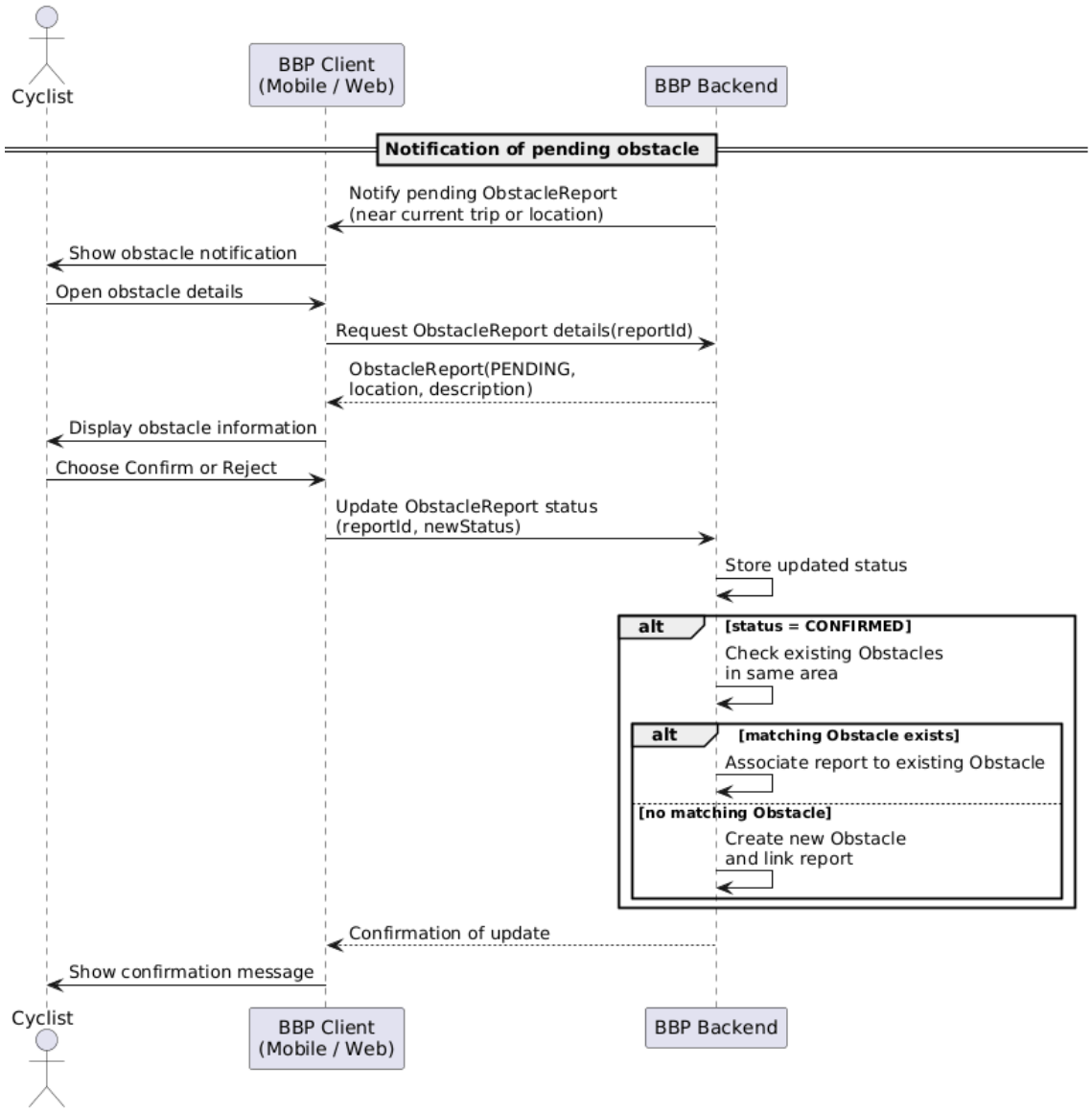


Figure 3.10: Validate detected obstacle - Use Case Diagram

UC07: Browse bike paths and conditions

Actors:	Cyclist (registered or unregistered user)
Entry Conditions:	• The BBP system is reachable. • At least one BikePath exists in the system.
Flow of Events:	1. The cyclist navigates to the "Bike Paths" section of the application. 2. BBP requests the list of BikePaths from the backend. 3. The backend returns a list of available paths, each with: • origin and destination, • aggregated status, • score, • number of obstacles or reports. 4. The cyclist selects a specific BikePath from the list. 5. BBP requests detailed information for the selected path. 6. The backend returns detailed data such as: • its composing Streets, • existing Obstacles on those streets, • aggregated PathStatusReports. 7. BBP displays the complete path information to the cyclist.
Exit condition:	The cyclist visualizes detailed information of the selected BikePath.
Exceptions:	1. No BikePaths exist: BBP displays an empty state message. 2. Map or backend services unavailable: BBP shows an error and suggests retrying. 3. Some Street or Obstacle information cannot be retrieved: BBP displays partial data with a warning.

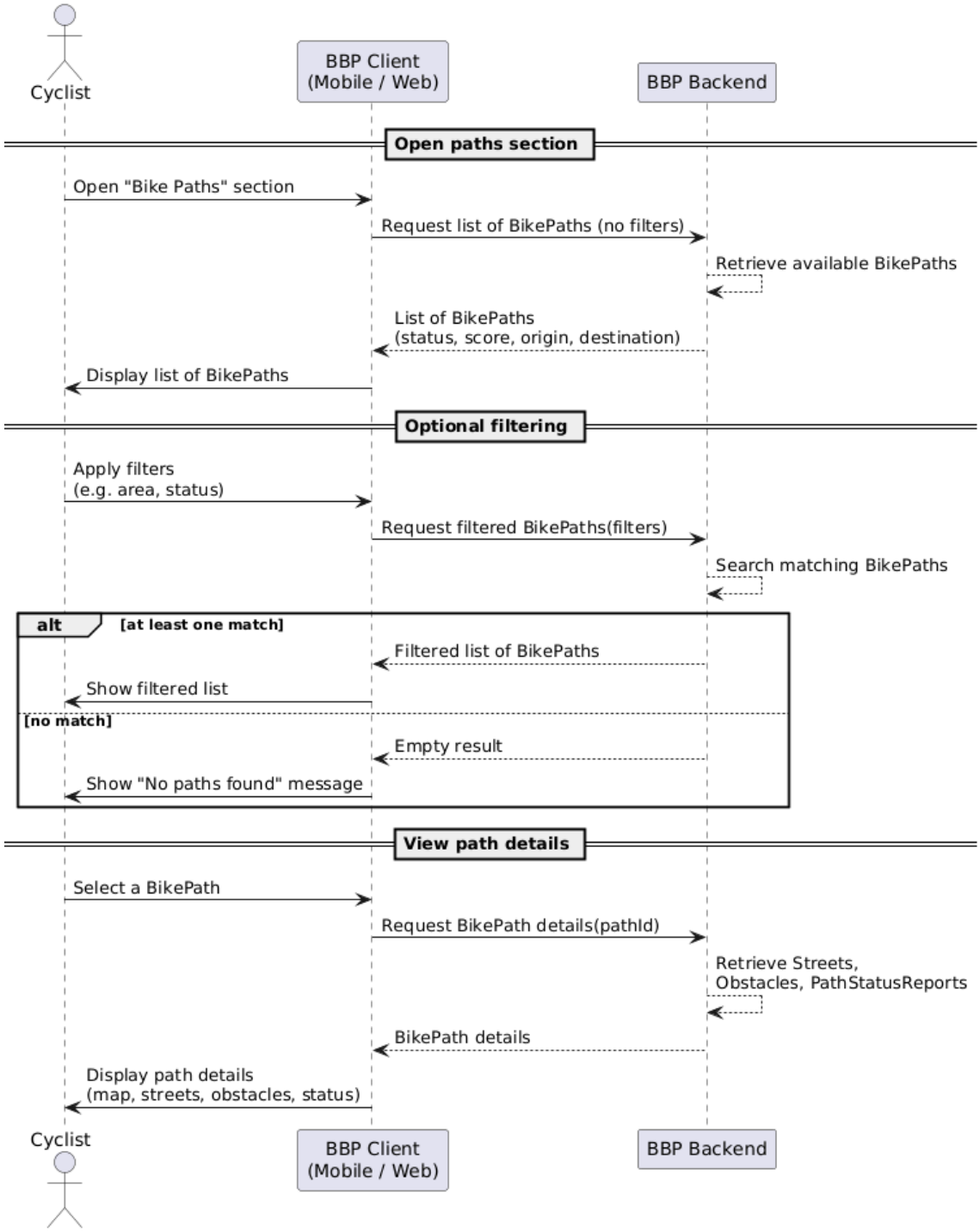


Figure 3.11: Submit obstacle report - Sequence Diagram

UC08: Submit path status report

Actors:	Cyclist (registered user)
Entry Conditions:	• The cyclist is authenticated in the BBP system. • At least one `BikePath` is available in the system.
Flow of Events:	1. The cyclist navigates to the "Bike Paths" section and selects a specific BikePath. 2. BBP displays detailed information of the selected path (streets, status, known obstacles). 3. The cyclist chooses the option *Rate path condition*. 4. BBP shows a form where the cyclist can: • select a `PathStatus` value (e.g., OPTIMAL, MEDIUM, SUFFICIENT, REQUIRES_MAINTENANCE); • optionally add comments; • choose whether the report should be publishable to other users. 5. The cyclist fills in the form and confirms the submission. 6. BBP sends the path status report data to the backend. 7. The backend creates a new `PathStatusReport` associated with: • the selected BikePath, • the current cyclist, • the selected PathStatus, • the acquisition mode (MANUAL or AUTOMATIC, if derived from a trip), • the chosen `isPublishable` setting, • the current timestamp as `createdAt`. 8. The backend updates the aggregated status or score of the BikePath based on the new report. 9. BBP displays a confirmation message and the updated status (if visible to the cyclist).
Exit condition:	A new `PathStatusReport` is stored and linked to the selected BikePath and cyclist; the path's aggregated status/score is updated.

Exceptions:	1. The cyclist does not select any status value: BBP highlights the missing field and does not submit the report. 2. The selected BikePath no longer exists or cannot be retrieved: BBP shows an error and returns to the path list. 3. The backend is temporarily unavailable: BBP notifies the cyclist and suggests retrying later; no report is stored.

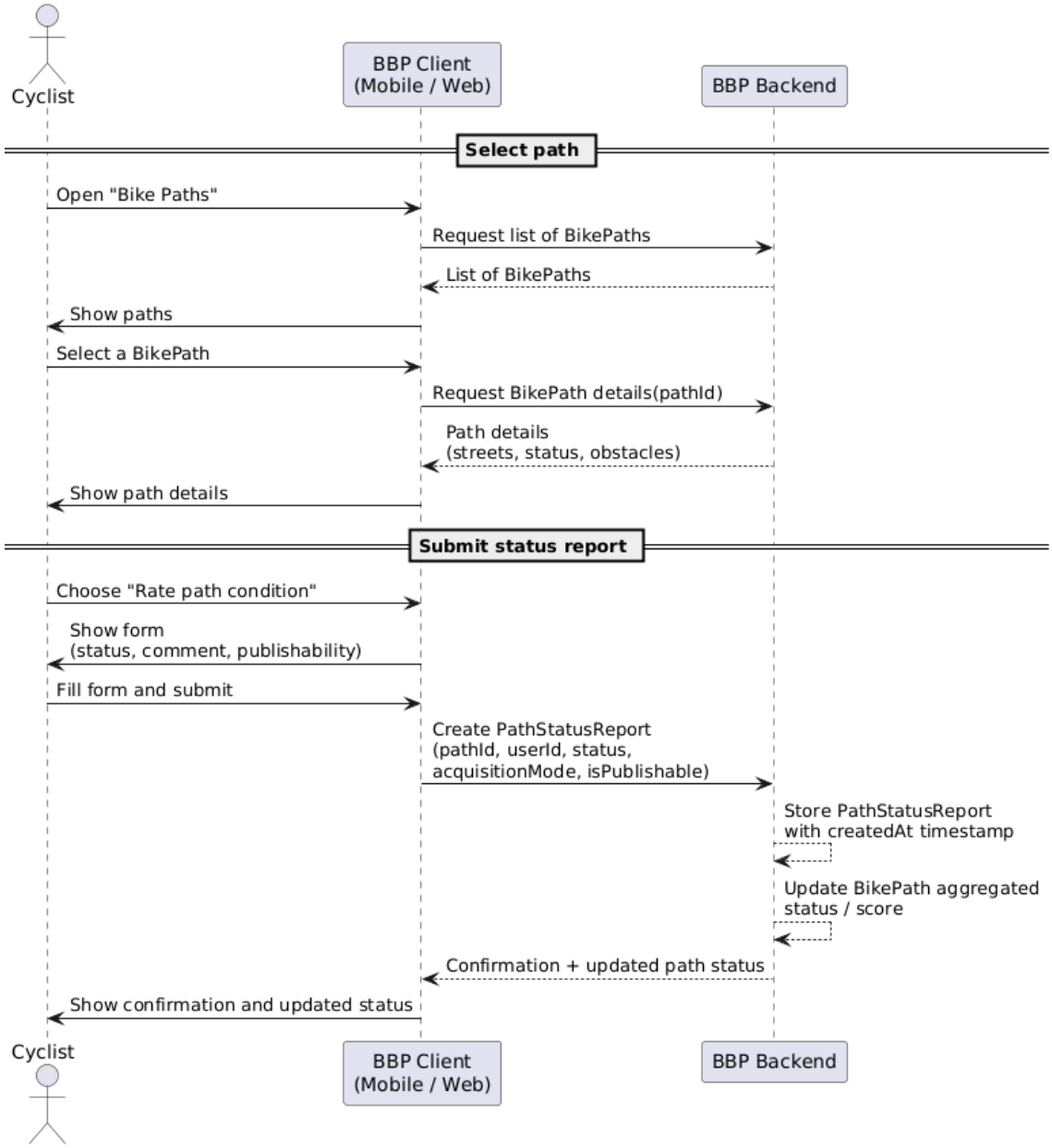


Figure 3.12: Submit obstacle report - Sequence Diagram

UC09: Manage data publishability

Actors:	Cyclist (registered user)
Entry Conditions:	• The cyclist is authenticated in the BBP system. • The cyclist has at least one contribution stored in the system (e.g., ObstacleReport or PathStatusReport).
Flow of Events:	1. The cyclist navigates to the *My contributions* section of the application. 2. BBP requests from the backend the list of contributions associated with the cyclist, including their current publishability status. 3. The backend returns the list of contributions (e.g., obstacle reports and path status reports) with their `isPublishable` flag. 4. BBP displays the contributions along with controls (e.g., toggles or checkboxes) for changing publishability. 5. The cyclist changes the publishability setting for one or more contributions. 6. The cyclist confirms the changes. 7. BBP sends the updated publishability choices to the backend. 8. The backend updates the `isPublishable` field for the specified contributions. 9. BBP displays a confirmation that the publishability settings have been updated.
Exit condition:	The `isPublishable` flags of the selected contributions are updated according to the cyclist's choices. Private items are no longer visible to other users.
Exceptions:	1. No contributions exist: BBP displays an empty state message indicating there is nothing to manage. 2. Backend temporarily unavailable: BBP notifies the cyclist that changes could not be saved and suggests retrying later. 3. A specific contribution cannot be updated (e.g., already deleted or invalid): BBP shows an error for that item and leaves its previous status unchanged.

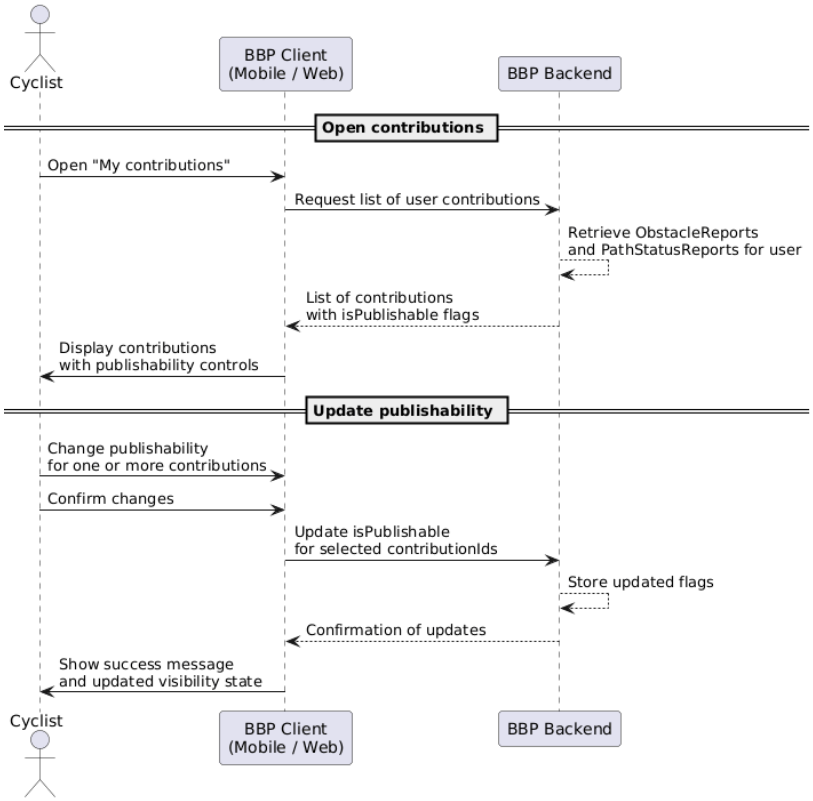


Figure 3.13: Manage data publishability - Use Case Diagram

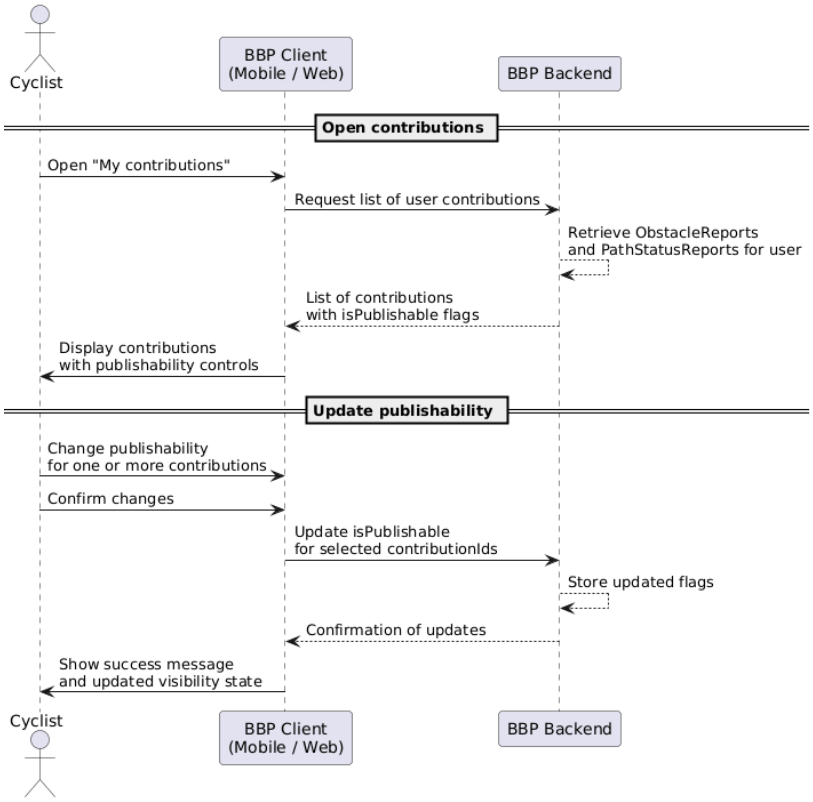


Figure 3.14: Manage data publishability - Sequence Diagram

3.2.4. Goals - Requirements Mapping

The following table summarizes how each high-level goal of BBP is supported by one or more functional requirements listed in Table 3.1. Each requirement identifier (R01, R02, . . . , R35) refers to the corresponding entry in the Requirements List.

Goal	Related Requirements
G01	R01, R02
Allow the registered user (RU) to create and manage a personal account.	
G02	R04, R05, R06, R07, R08, R09, R27, R28, R33, R34
Allow the RU to record private personal trips (manual or automatic).	
G03	R07, R28
Allow the RU to view detailed statistics for each trip (distance, speed, performance metrics).	
G04	R08, R10, R20, R21, R22, R24

Allow the RU to manually insert path information by selecting streets and specifying their status.	
G05	R04, R05, R06, R33, R34
Allow the RU to collect automatic trip data via mobile sensors (GPS, accelerometer, gyroscope).	
G06	R11, R13, R14, R15, R16, R18, R19, R35
Allow the RU to confirm or correct automatically inferred obstacles or street conditions.	
G07	R20, R21, R22, R23, R24, R25, R29, R30
Allow the RU to publish, edit, or delete paths they contributed (and control their visibility).	
G08	R07, R09, R27, R28
Allow the RU to see a dashboard summarizing their cycling activity.	
G09	R03, R20, R21, R24, R25, R26
Allow users to browse all published paths and their associated conditions.	
G10	R20, R25
Allow users to search for paths based on origin and destination.	
G11	R20, R26, R32, R17, R18
Allow users to view map representations of possible routes, including streets and obstacles.	
G12	R21, R24, R25, R26
Allow users to sort or filter paths based on score or conditions.	
G13	R07, R21, R24
Enable the system to compute the score of a trip/path based on its characteristics.	
G14	R09, R31
Enable the system to enrich trips with meteorological information from an external weather service.	
G15	R05, R11, R13, R14, R15, R33, R34, R35
Enable the system to detect anomalies such as potholes using sensor data and related reports.	
G16	R20, R21, R24, R26, R28
Enable the system to merge similar paths contributed by multiple users, considering freshness and disagreement handling.	

G17	R20, R26, R32
Enable the system to provide map data through a mapping API (e.g., OpenStreetMap).	

Table 3.12: Goals to Requirements Mapping Table

3.3. Performance Requirements

Latency Requirements Regarding performance, BBP must be developed with a scalable architecture capable of assuring its users a high Quality of Experience (QoE). In particular, the latency for real-time interactions—such as starting/stopping a trip recording or submitting an obstacle report—should be less than 300ms. Map visualization and route queries, which involve complex geospatial calculations, should load within 2 seconds under normal network conditions.

Storage Requirements The system must safely store geospatial data and user contributions. Unlike text-based systems, BBP deals with high-frequency sensor logs. The most storage-intensive objects will be the sequences of `TrackPoints` (GPS coordinates) for recorded trips and images associated with obstacle reports (e.g., photos of potholes). An average of 50 MB/user should be allocated to account for these data types, ensuring the scalability of the application.

Architecture Sizing The primary market for BBP consists of urban cyclists and commuters. Assuming a launch in a major metropolitan area with a potential user base of 50,000 active daily cyclists, the system must be designed to handle high concurrency during peak commuting hours (typically 08:00-09:00 and 17:00-18:00). The infrastructure must be dynamically scalable, capable of adjusting to accommodate usage spikes during weekends or cycling events.

3.4. Design Constraints

3.4.1. Standards Compliance

BBP must strictly comply with EU's GDPR regulations, which is particularly critical given the system's collection of precise geolocation data[cite: 95]. Users must explicitly grant permissions for location tracking, and the system must allow for the anonymization or deletion of trip history upon request. Furthermore, map data integration should comply with the usage policies of the chosen external map provider (e.g., OpenStreetMap).

3.4.2. Hardware Limitations

Server-side: The backend must be able to run on standard cloud infrastructure (commodity x86 hardware) to ensure cost-effectiveness and scalability.

Client-side: Unlike typical web applications, BBP has specific hardware dependencies for its core functionality. The "Automated Mode" requires a smartphone equipped with a functioning **GPS receiver**, **accelerometer**, and **gyroscope**[cite: 95]. Consequently, the primary client interface must be a native mobile application (or a Progressive Web App with sensor access) capable of running background processes for data acquisition without draining the device battery excessively.

3.5. Software System Attributes

3.5.1. Reliability

The system must be designed in such a way that errors can't propagate over the whole architecture and fault can be easily identified and isolated. Enough replicas of data and services should be implemented to ensure the reliability of the application.

3.5.2. Availability

The system must be available 99.99% (four-nines) of the time, thus allowing 52 minutes of down-time yearly.

Maintenance should be carried out when the impact on the clients is negligible and must be announced in advance.

3.5.3. Security

The system implements a comprehensive authentication and authorization framework to manage user access rights. Authentication verifies the identity of users attempting to log in, while authorization ensures that authenticated users possess appropriate permissions for requested actions.

To safeguard the system's integrity, standard security measures will be implemented, including encryption of user credentials and personal data and protection mechanisms against database query injection vulnerabilities.

3.5.4. Maintainability

The system must be modular enough to allow future expansions with limited effort and repercussion. Enough replicas of the system must be always up to allow maintenance without impacting on user's traffic.

3.5.5. Portability

The system architecture ensures portability through a clear separation of concerns. The server-side components run on standard x86 hardware or cloud containers, making them platform-independent. The client-side functionality is split: the *Dashboard* and browsing features are accessible via any standard web browser, while the *Trip Tracker* requires a mobile environment to access specific hardware sensors (GPS, Accelerometer).

4 | Formal Analysis using Alloy

This chapter presents the formal modeling and verification of the *Best Bike Paths* (BBP) system using the Alloy specification language and the Alloy Analyzer. The purpose of this analysis is to complement the natural language requirements and semi-formal UML diagrams with a precise, analyzable model.

Alloy is employed here as a lightweight formal method to reason about the system's structure and critical behaviors. Specifically, the analysis focuses on:

- **Trip Coherence:** Ensuring chronological correctness of recorded trips and track points.
- **Data Consistency:** Validating the relationships between obstacles, reports, and streets.
- **Aggregation Logic:** Verifying the majority-voting mechanism used to determine the status of bike paths.
- **Workflow Validation:** Enforcing valid state transitions for obstacle reporting (e.g., from detection to confirmation).

4.1. Alloy Model

The Alloy model captures the static structure and dynamic constraints of the BBP domain. The model is presented below, divided into logical segments for clarity.

4.1.1. Signatures and Enums

We begin by defining the basic data types, enumerations, and core entities such as Users, DateTime, and GeoPoints. These signatures form the vocabulary of our model.

```
// USERS
sig RegisteredUser {}

// ENUMERATIONS
```

```
abstract sig PathStatus {}
one sig OPTIMAL, MEDIUM, SUFFICIENT, REQUIRES_MAINTENANCE extends
   PathStatus {}

abstract sig AcquisitionMode {}
one sig MANUAL, AUTOMATIC extends AcquisitionMode {}

abstract sig WeatherStatus {}
one sig SUNNY, RAINY, CLOUDY, SNOWY extends WeatherStatus {}

abstract sig ObstacleStatus {}
one sig PENDING, CONFIRMED, REJECTED extends ObstacleStatus {}

// GEOPOINT, TIME, TRACKPOINT
sig GeoPoint {
}

sig  DateTime {
    timestamp: one Int,
}

// Time is a strict partial order
fact timeOrder {
    all t1, t2 :  DateTime |
    t1 != t2 implies t1.timestamp != t2.timestamp
}

sig TrackPoint {
    timestamp   : one  DateTime,
    coordinates : one GeoPoint,
    elevation   : one Int
}
```

4.1.2. Domain Entities: Trips, Weather, and Reports

The following signatures model the core domain concepts. `Trip` aggregates track points and weather data, while `Report` signatures handle user contributions regarding obstacles and path quality.

```
// WEATHER CONDITIONS
```

```
sig WeatherConditions {
    temperature : one Int,
    windSpeed   : one Int,
    status      : one WeatherStatus
}

// STREETS, OBSTACLES, BIKE PATHS

sig Street {
    var status  : one PathStatus,
    obstacles   : set Obstacle
}

sig Obstacle {
    coordinates : one GeoPoint,
    street      : one Street
}

sig BikePath {
    origin      : one GeoPoint,
    destination : one GeoPoint,
    var score   : one Int,
    var status  : one PathStatus,
    streets     : some Street,
    pathReports : set PathStatusReport
}

// REPORTS
sig PathStatusReport {
    createdAt       : one  DateTime,
    status          : one PathStatus,
    acquisitionMode : one AcquisitionMode,
    by              : one RegisteredUser,
    path            : one BikePath
}

sig ObstacleReport {
    detectedAt      : one  DateTime,
    acquisitionMode : one AcquisitionMode,
```

```
    status          : one ObstacleStatus,
    by              : one RegisteredUser,
    trip            : lone Trip,
    obstacle        : lone Obstacle
}

// TRIPS
sig Trip {
    startTime       : one  DateTime,
    endTime         : one  DateTime,
    distance        : one Int,
    avgSpeed        : one Int,
    acquisitionMode : one AcquisitionMode,
    owner           : one RegisteredUser,
    bikePath        : lone BikePath,
    track           : seq TrackPoint,
    weather         : lone WeatherConditions,
    obstacleReports : set ObstacleReport
}
```

4.1.3. System Facts and Constraints

The logic of the system is enforced through facts. These constraints ensure that the model instances strictly adhere to the requirements.

Structural Consistency: The following facts ensure that track points are correctly ordered within a trip, and that objects (like obstacles and reports) are correctly linked to their parents.

```
// Track points are temporally ordered according to their position
    in the sequence
fact trackPointOrder {
    all t : Trip |
        all i, j : Int |
            i >= 0 and j >= 0 and
            i < #t.track and j < #t.track and
            i < j
            implies
            t.track[i].timestamp.timestamp < t.track[j].timestamp.
               timestamp
```

```
}

// Each track point belongs to exactly one trip
fact trackPointOwnership {
    all tp : TrackPoint |
        one t : Trip | some i : Int | tp = t.track[i]
}

fact automaticTripsHaveTrack {
    all t : Trip |
        t.acquisitionMode = AUTOMATIC implies #t.track > 0
}

fact manualTripsNoTrack {
    all t : Trip |
        t.acquisitionMode = MANUAL implies #t.track = 0
}

fact pathReportConsistency {
    all r : PathStatusReport | r in r.path.pathReports
}

fact obstacleStreetConsistency {
    all o : Obstacle | o in o.street.obstacles
}

fact obstacleReportTripConsistency {
    all o : ObstacleReport |
        o.trip != none implies o in o.trip.obstacleReports
}

fact weatherConditionsOwnership {
    all w : WeatherConditions | lone t : Trip | t.weather = w
}
```

Business Logic and Merging: Crucial business rules, such as the aggregation of path status based on majority voting and the lifecycle of obstacle reports (only confirmed reports link to actual obstacles), are defined here.

```
// Obstacle Workflow Logic
```

```
fact confirmedReportsLinkToObstacle {
    all o : ObstacleReport |
        o.status = CONFIRMED implies some o.obstacle
}

fact notConfirmedNoObstacleLink {
    all o : ObstacleReport |
        o.status != CONFIRMED implies no o.obstacle
}

fact obstacleReportsWithoutTripAreManual {
    all o : ObstacleReport |
        no o.trip implies o.acquisitionMode = MANUAL
}

fact nonNegativeTripMetrics {
    all t : Trip |
        t.avgSpeed >= 0 and
        t.distance >= 0
}

fact nonNegativeTimestamps {
    all dt : DateTime | dt.timestamp >= 0
}
// Path Status Merging (Majority Rule)
pred isMajorityStatus[p : BikePath, ps : PathStatus] {
    all ps2 : PathStatus |
        #{ r : p.pathReports | r.status = ps } >= #{ r : p.
           pathReports | r.status = ps2 }
}

fact bikePathStatusMerging {
    all p : BikePath |
        some p.pathReports implies isMajorityStatus[p, p.status]
}

fact defaultPathStatus {
    all p : BikePath |
        no p.pathReports implies some p.status
```

```
}
fact tripTimeCoherence {
    all t : Trip |
        // start prima di end
        t.startTime.timestamp < t.endTime.timestamp and

        // ogni trackpoint sta dentro l'intervallo del trip
        all i : Int |
            i >= 0 and i < #t.track implies {
                let tp = t.track[i] |
                    t.startTime.timestamp < tp.timestamp.timestamp
                        and
                    tp.timestamp.timestamp < t.endTime.timestamp
            }
}

fact obstacleReportTimeBounds {
    all o : ObstacleReport |
        some o.trip implies {
            let t = o.trip |
                t.startTime.timestamp < o.detectedAt.timestamp and
                o.detectedAt.timestamp < t.endTime.timestamp
        }
}
```

4.1.4. Assertions and Predicates

To verify the model, we define assertions (properties that must always hold) and predicates (scenarios we wish to generate).

```
//////////////////////////////////////////////////////////
// ASSERTIONS
//////////////////////////////////////////////////////////

assert confirmedMeansObstacle {
    all o : ObstacleReport | o.status = CONFIRMED implies some o.
       obstacle
}

assert automaticTripsHaveTrackPoints {
```

```
    all t : Trip |
        t.acquisitionMode = AUTOMATIC implies #t.track > 0
}

assert tripTimesAreConsistent {
    all t : Trip |
        t.startTime.timestamp < t.endTime.timestamp
}

assert bikePathStatusIsMajority {
    all p : BikePath |
        some p.pathReports implies
            isMajorityStatus[p, p.status]
}

assert manualTripsHaveNoTrackPoints {
    all t : Trip |
        t.acquisitionMode = MANUAL implies #t.track = 0
}

////////////////////////////////////////////////////////
// SHOW PREDICATES (Scenario Generation)
////////////////////////////////////////////////////////

pred showAutomaticTrip {
    some t : Trip |
        t.acquisitionMode = AUTOMATIC and
        #t.track > 2 and
        some t.weather
}

pred showObstacleWorkflow {
    some p : ObstacleReport | p.status = PENDING
    some c : ObstacleReport | c.status = CONFIRMED
    some r : ObstacleReport | r.status = REJECTED
}

pred showPathMerging {
    some p : BikePath |
```

```
        #p.pathReports > 2
}

pred showCompleteScenario {
    #RegisteredUser > 2
    some t1 : Trip | t1.acquisitionMode = AUTOMATIC and #t1.track
       > 2 and some t1.weather
    some t2 : Trip | t2.acquisitionMode = MANUAL
    some c : ObstacleReport | c.status = CONFIRMED and some c.
       obstacle
    some bp : BikePath | #bp.pathReports > 1
}

pred majorityShouldWin {
    some p : BikePath |
        #p.pathReports = 3 and
        #{ r : p.pathReports | r.status = OPTIMAL } = 2 and
        #{ r : p.pathReports | r.status != OPTIMAL } = 1 and
        p.status = OPTIMAL
}
```

4.2. Alloy Results

The model was analyzed using the Alloy Analyzer. All the defined assertions, including those concerning trip temporal coherence, acquisition-mode constraints, and majority-based bike path status aggregation, were verified using the `check` command with a finite scope. No counterexamples were found, increasing confidence in the correctness and internal consistency of the model.

Furthermore, the `run` commands defined above were executed to generate visual instances of key system scenarios. These instances confirm that the model permits the intended behaviors described in the requirements.

Scenario 1: Automatic Trip Recording. Figure 4.1 presents a consistent instance generated by the `showAutomaticTrip` predicate. The visualization focuses on `Trip0`, an automatic trip recorded by `RegisteredUser1`.

The instance validates the following structural and temporal constraints defined in the model:

- **Track Sequence Continuity:** In accordance with the `automaticTripsHaveTrack` fact, `Trip0` is associated with a sequence of track points (`TrackPoint0`, `TrackPoint2`, `TrackPoint1`) indexed via the `track` relation. The model enforces that these points are strictly ordered by time based on their sequence index.
- **Environmental Context:** The trip is successfully linked to external environmental data. The relation `weather` connects the trip to a `WeatherConditions` atom with a `CLOUDY` status, satisfying the requirement that automatic trips can capture meteorological conditions.
- **Infrastructure Interaction:** The trip exists alongside a `BikePath` atom (status `OPTIMAL`), demonstrating the system's ability to model user trips within the context of the existing path inventory.

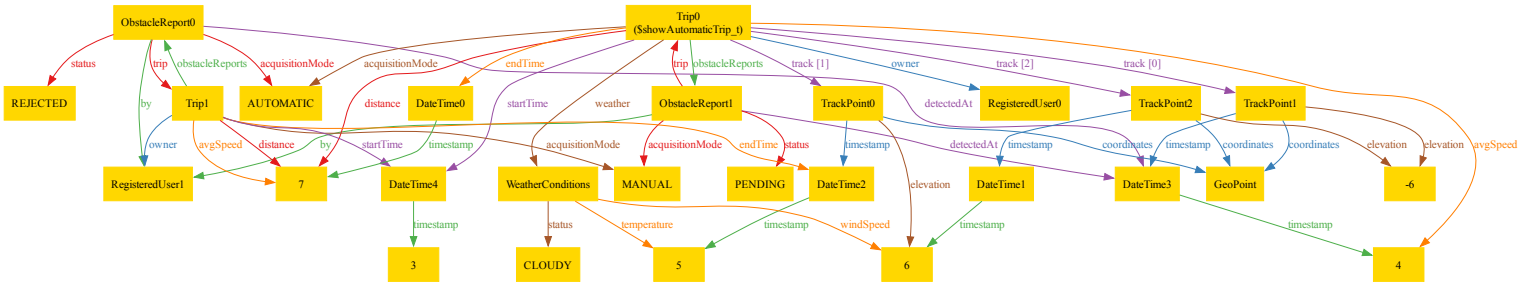


Figure 4.1: Generated image for the `showAutomaticTrip` predicate

Scenario 2: Obstacle Reporting Workflow. Figure 4.2 displays an instance generated by the `showObstacleWorkflow` predicate. This scenario validates the system's ability to handle the three distinct states of obstacle reporting and their differing structural implications.

The diagram highlights three specific reports corresponding to the assertion logic:

- **Confirmed State (Validated Hazard):** `ObstacleReport2` is marked with the status `CONFIRMED`. Consistent with the `confirmedReportsLinkToObstacle` fact, this report is the only one in the set linked to a concrete `Obstacle` atom. This signifies that the system has verified the existence of the hazard on the associated `Street`.
- **Pending State (Awaiting Verification):** `ObstacleReport4` is shown with the status `PENDING`. It represents a newly submitted report that is currently stored in the system but has not yet been linked to a confirmed `Obstacle` entity.

- **Rejected State (False Positive):** `ObstacleReport3`, submitted by `RegisteredUser1`, holds the status `REJECTED`. As required by the `notConfirmedNoObstacleLink` fact, this report is isolated from any `Obstacle` atom, ensuring that invalid reports do not pollute the map data.

The scenario also demonstrates the integration of these reports within the broader environment, showing active `BikePaths` (e.g., `BikePath0`) and concurrent environmental factors.

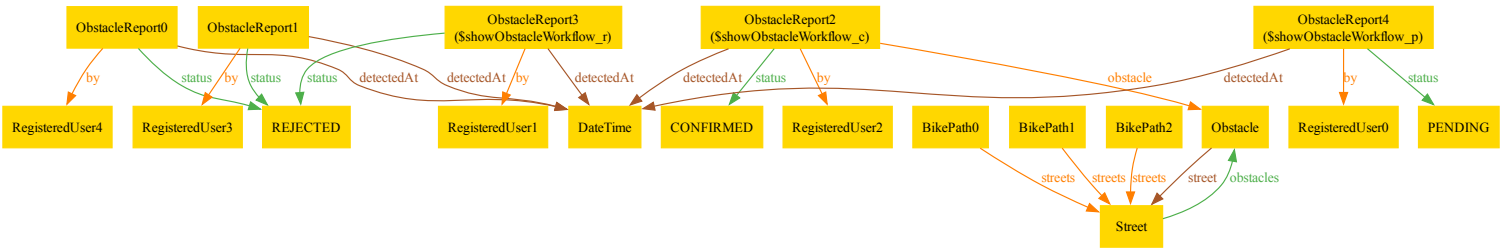


Figure 4.2: Obstacle workflow instance showing the coexistence of Pending, Confirmed, and Rejected reports, where only the confirmed report is linked to a physical Obstacle.

Scenario 3: Path Status Merging. Figure 4.3 illustrates the aggregation logic generated using the `showPathMerging` predicate, while enforcing the global bikePathStatusMerging constraint. The central entity is `BikePath2`, which is associated with multiple conflicting reports submitted by users.

The instance demonstrates the "Majority Rule" logic defined in the system's requirements:

- **Conflicting Inputs:** The path is linked to three distinct reports via the `pathReports` relation:
  - `PathStatusReport0` and `PathStatusReport2` both indicate the status is `SUFFICIENT`.
  - `PathStatusReport1` indicate the status is `MEDIUM`.
- **Majority Resolution:** Consistent with the `bikePathStatusMerging` fact, the system calculates the mode of the submitted reports. Since "Sufficient" has the majority count (2 vs 1), `BikePath2` is assigned the status `SUFFICIENT`.
- **Data Integrity:** All reports are correctly linked to a `RegisteredUser` and a `DateTime` timestamp, ensuring that the merging algorithm operates on valid, traceable data entries.

The scenario also confirms that this logic operates correctly within a larger context, as `BikePath2` coexists with other paths (`BikePath0`, `BikePath1`) and global `WeatherConditions` (shown as `SNOWY`), without structural conflicts.

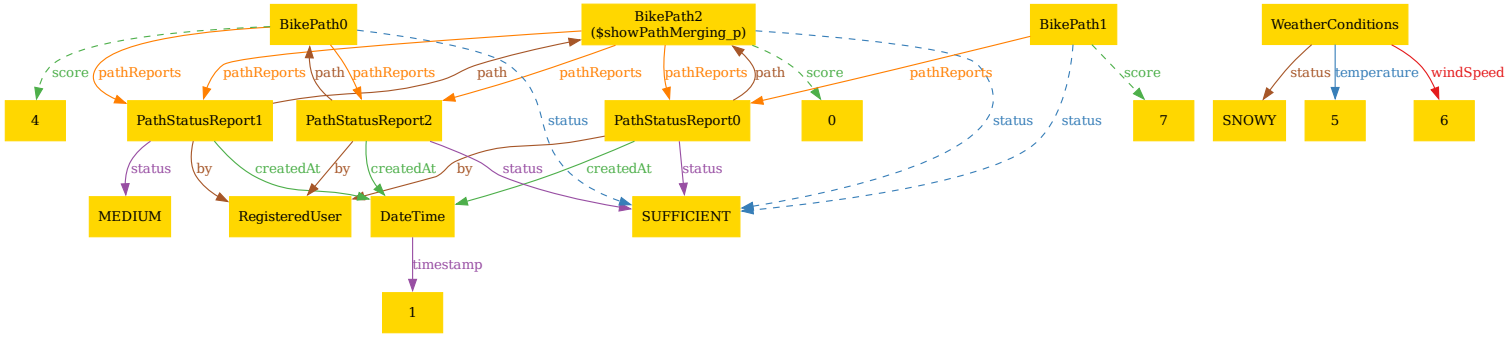


Figure 4.3: Path Merging instance showing `BikePath2` resolving a status of MEDIUM based on the majority of incoming user reports.

Scenario 4: Comprehensive System State.

Figure 4.4 depicts a complete world generated by the `showCompleteScenario` predicate, capturing the simultaneous presence and interaction of trips, obstacle reporting, path status evaluation, and environmental conditions within a single consistent system state.

The instance highlights the following integrated aspects:

- **Dual Acquisition Modes Across Multiple Entities:** Both `MANUAL` and `AUTOMATIC` acquisition modes coexist and are applied consistently across different system components.
  - `Trip1` is an `AUTOMATIC` trip, associated with weather information and enriched with an ordered sequence of GPS track points (`TrackPoint0`, `TrackPoint1`, `TrackPoint2`), each providing spatial, temporal, and elevation data.
  - `Trip0` is a `MANUAL` trip. It records ownership, temporal bounds, distance, and average speed, but, consistently with the model constraints, it is not associated with any track points.
- **Obstacle Reporting Lifecycle:** The system includes multiple `ObstacleReport` instances with different acquisition modes and statuses.
  - `ObstacleReport0`, acquired automatically during `Trip1`, is marked as `CONFIRMED` and is correctly linked to a physical `Obstacle` located on a `Street`.

- `ObstacleReport1` is a manually submitted report with status `REJECTED`.
- `ObstacleReport2` is also manually submitted and remains in the `PENDING` state.

All obstacle reports are associated with a registered user and explicit detection timestamps.

- **Path Status Reporting and Aggregation:** A single `BikePath` is defined by an origin and a destination `GeoPoint` and is associated with the same `Street` hosting the obstacle. The bike path receives multiple `PathStatusReport` instances:
  - `PathStatusReport0` is acquired automatically and reports a `MEDIUM` status.
  - `PathStatusReport1` is acquired manually and reports a `SUFFICIENT` status.

  These reports are created by a registered user and include creation timestamps. The bike path aggregates this information into a single path-level status of `MEDIUM`, along with a numerical score.
- **Environmental Context:** The system state also includes environmental conditions through a `WeatherConditions` instance with status `SNOWY`, temperature, and wind speed. These conditions are explicitly linked to `Trip1`, demonstrating the integration of contextual data into trip-level information.

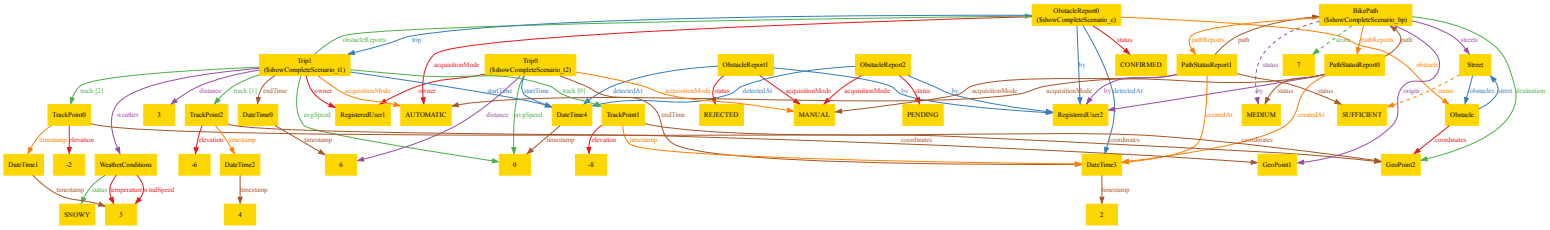


Figure 4.4: Complete scenario instance showing the coexistence of automatic and manual trips, obstacle reporting with different validation states, path status aggregation, and environmental conditions.

Scenario 5: Path Status Majority Voting.

Figure 4.5 illustrates the instance generated by the `majorityShouldWin` predicate, which is designed to validate the resolution of conflicting path status reports through a majority-based aggregation strategy.

The diagram focuses on `BikePath1`, which is associated with three independent path status reports:

- **Conflicting Status Reports:**
  - `PathStatusReport1` and `PathStatusReport2` both assign the status `OPTIMAL` to the path.
  - `PathStatusReport0` assigns a lower-quality status, namely `MEDIUM`.
- **Majority-Based Aggregation Result:** According to the majority voting logic enforced by the model, the system selects the most frequently reported status among those associated with the path. Since `OPTIMAL` appears twice out of three reports, `BikePath1` is assigned the aggregated status `OPTIMAL`, together with a corresponding numerical score.
- **Heterogeneous Data Sources:** The three reports originate from different registered users (`RegisteredUser0` and `RegisteredUser1`) and employ heterogeneous acquisition modes:
  - `PathStatusReport0` and `PathStatusReport1` are acquired in `AUTOMATIC` mode.
  - `PathStatusReport2` is acquired in `MANUAL` mode.

  This demonstrates that the aggregation mechanism operates independently of both the reporting user and the acquisition modality.
- **Contextual Information:** For completeness, the scenario also includes environmental context through a `WeatherConditions` instance with status `SNOWY`, as well as a `Street` associated with the bike paths. These elements remain orthogonal to the voting process and do not influence the outcome of the majority decision.

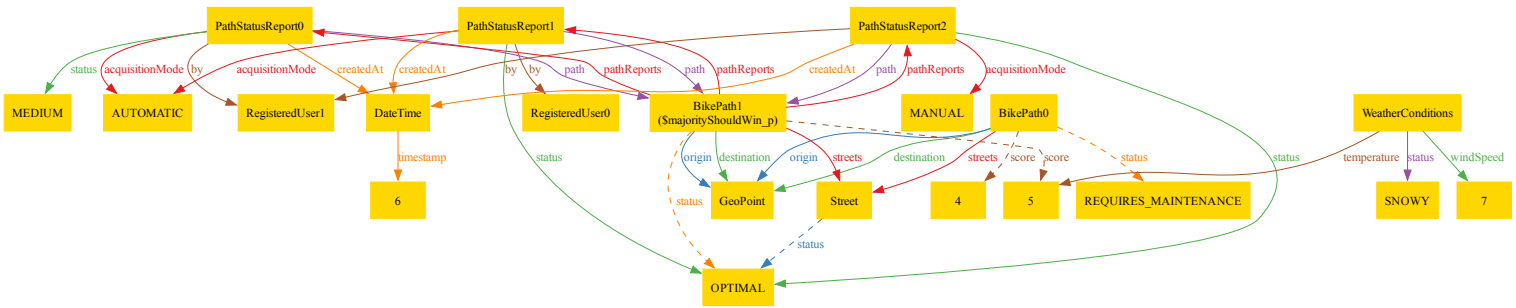


Figure 4.5: Majority voting scenario in which two `OPTIMAL` reports override a single `MEDIUM` report, resulting in an aggregated `OPTIMAL` status for the bike path.

5 | Effort Spent

Group Member	Effort Spent	
Giuseppe	Introduction	*2 h*
	Overall Description	*4 h*
	Specific Requirements	*8 h*
	Formal Analysis	*6 h*
	Reasoning	*5 h*
Lorenzo	Introduction	*2 h*
	Overall Description	*4 h*
	Specific Requirements	*9 h*
	Formal Analysis	*7.5 h*
	Reasoning	*6 h*
Giacomo	Introduction	*2 h*
	Overall Description	*4 h*
	Specific Requirements	*7.5 h*
	Formal Analysis	*9 h*
	Reasoning	*5 h*

Table 5.1: Effort Spent by Each Member of the group

6 References

6.1. References

- ISO/IEC/IEEE 29148:2018 - Systems and software engineering - Life cycle processes - Requirements engineering.
- The Requirement Engineering and Design Project specification document A.Y. 2025-2026.
- Lecture slides and notes from Prof. Di Nitto's course
- `https://github.com/MarcoApollonio02/ApollonioBossiChiroli`

6.2. Tools and Technologies

- **GitHub** – Used for version control and project sharing.
- **LaTeX** and **Overleaf** – Adopted as the typesetting system and collaborative editor for drafting the document.
- **PlantUML** (*editor.plantuml.com*) – Utilized for designing UML class and sequence diagrams.
- **bpmn.io** – Employed for the creation of BPMN diagrams.
- **PDF24** – Employed for the conversion of BPMN diagrams to pdf format.
- **Alloy** – Used for formal analysis and logic modeling.
- **ChatGPT** and **Gemini** – Consulted for English language refinement, typo correction, and proofreading.